PUC Minas



Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Sumário

- Testes de resiliência dos serviços
  - Teste unitário
  - Teste de desempenho
  - Teste de carga

Testes de resiliência dos serviços

- Os testes desempenham um papel fundamental no desenvolvimento de software, proporcionando uma série de benefícios que vão além da simples detecção de erros (ou bugs).
- Eles são uma prática essencial para garantir a **qualidade**, **confiabilidade** e **manutenibilidade** de um sistema.



Testes de resiliência dos serviços

- Nos últimos anos, **testar** se tornou motivo de aprovação ou reprovação em etapas técnicas de processos seletivos para vagas de Desenvolvedor Jr.
    - Projetos que são entregues sem testes, muitas vezes são ignorados por recrutadores (tech leads, lideranças técnicas, etc).

- A importância dos testes no ciclo de vida do desenvolvimento de software se dá por vários motivos diferentes, vejamos os principais.

Testes de resiliência dos serviços

- **Detecção precoce de defeitos**:
  - Testes realizados desde as fases iniciais do desenvolvimento ajudam a identificar defeitos e falhas antes que eles se tornem problemas mais complexos e custosos de corrigir.

  - Detectar e corrigir problemas cedo no processo de desenvolvimento é mais eficiente e econômico.

Testes de resiliência dos serviços

- **Garantia de qualidade:**
  - Testes fornecem uma garantia de que o software está cumprindo os requisitos especificados.

  - Eles asseguram que as funcionalidades estão operando conforme esperado e que o sistema está em conformidade com os padrões de qualidade estabelecidos.

Testes de resiliência dos serviços

- **Manutenibilidade e evolução:**
  - Sistemas bem testados são mais fáceis de manter e evoluir.
  - Quando novos recursos são adicionados ou modificações são feitas, os testes garantem que as alterações não introduzam regressões ou comportamentos indesejados em partes existentes do software.

Testes de resiliência dos serviços

- **Redução de custos a longo prazo:**
  - Investir em testes pode inicialmente parecer um esforço adicional, mas a longo prazo, ajuda a evitar custos significativos relacionados à correção de bugs em produção, suporte técnico e possíveis perdas de clientes devido a problemas de qualidade.

Testes de resiliência dos serviços

- **Confiança e credibilidade:**
  - Um software testado e confiável constrói a confiança dos usuários e clientes.
  - A credibilidade de um sistema está diretamente relacionada à sua capacidade de fornecer uma experiência consistente e sem falhas.

Testes de resiliência dos serviços

- **Documentação viva:**
  - Conjuntos de testes servem como documentação viva do comportamento esperado do software.
  - Eles fornecem uma descrição detalhada das funcionalidades e podem servir como uma referência valiosa para desenvolvedores futuros.

Testes de resiliência dos serviços

- **Automatização para eficiência:**
  - A automação de testes permite a execução rápida e repetitiva de casos de teste, garantindo a consistência e economizando tempo.

  - Isso é especialmente valioso em ambientes ágeis, nos quais as mudanças são frequentes.

Testes de resiliência dos serviços

- **Maior satisfação do cliente:**
  - Um software que funciona corretamente e atende às expectativas dos usuários resulta em maior satisfação do cliente.
  - Isso fortalece a relação entre desenvolvedores e usuários finais.

Testes de resiliência dos serviços

- **Prevenção de problemas futuros:**
  - Além de identificar problemas existentes, os testes também ajudam a prever e prevenir problemas futuros.
  - Ao antecipar possíveis cenários de uso, os desenvolvedores podem criar um software mais robusto.

Testes de resiliência dos serviços

- **Equipe de QA (Qualidade assegurada)**
  - A equipe de garantia de qualidade (QA), em um time de desenvolvimento, desempenha um papel crucial para assegurar a entrega de software de alta qualidade.

  - A QA não se limita apenas a identificar bugs, mas abrange uma abordagem holística para garantir que o produto atenda aos mais altos padrões de desempenho, usabilidade e confiabilidade.

Testes de resiliência dos serviços

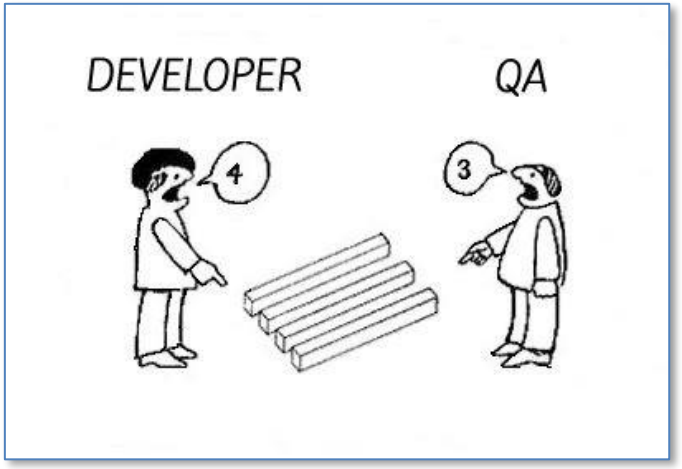
- **Equipe de QA (Qualidade assegurada)**
  - Uma equipe de QA é responsável por avaliar e melhorar continuamente os processos de desenvolvimento, além de garantir que o produto final atenda aos requisitos de qualidade estabelecidos.

  - A equipe trabalha em colaboração com os desenvolvedores para identificar e corrigir defeitos, otimizar processos e garantir que o software atenda às expectativas dos usuários finais.

Testes de resiliência dos serviços

- **Equipe de QA (Qualidade assegurada)**





Testes de resiliência dos serviços

- **Importância da equipe de QA (Qualidade assegurada)**
  - **Detecção precoce de defeitos**: Identificar e corrigir problemas durante as fases iniciais do desenvolvimento, economizando tempo e recursos.
  - **Garantia de qualidade**: Assegurar que o produto atenda a padrões de qualidade estabelecidos, resultando em maior satisfação do cliente.
  - **Eficiência operacional**: Automação de testes para aumentar a eficiência e permitir uma rápida validação de mudanças.
  - **Confiança do cliente**: Construir a confiança do cliente, fornecendo um produto livre de erros e de alta qualidade.

Testes de resiliência dos serviços

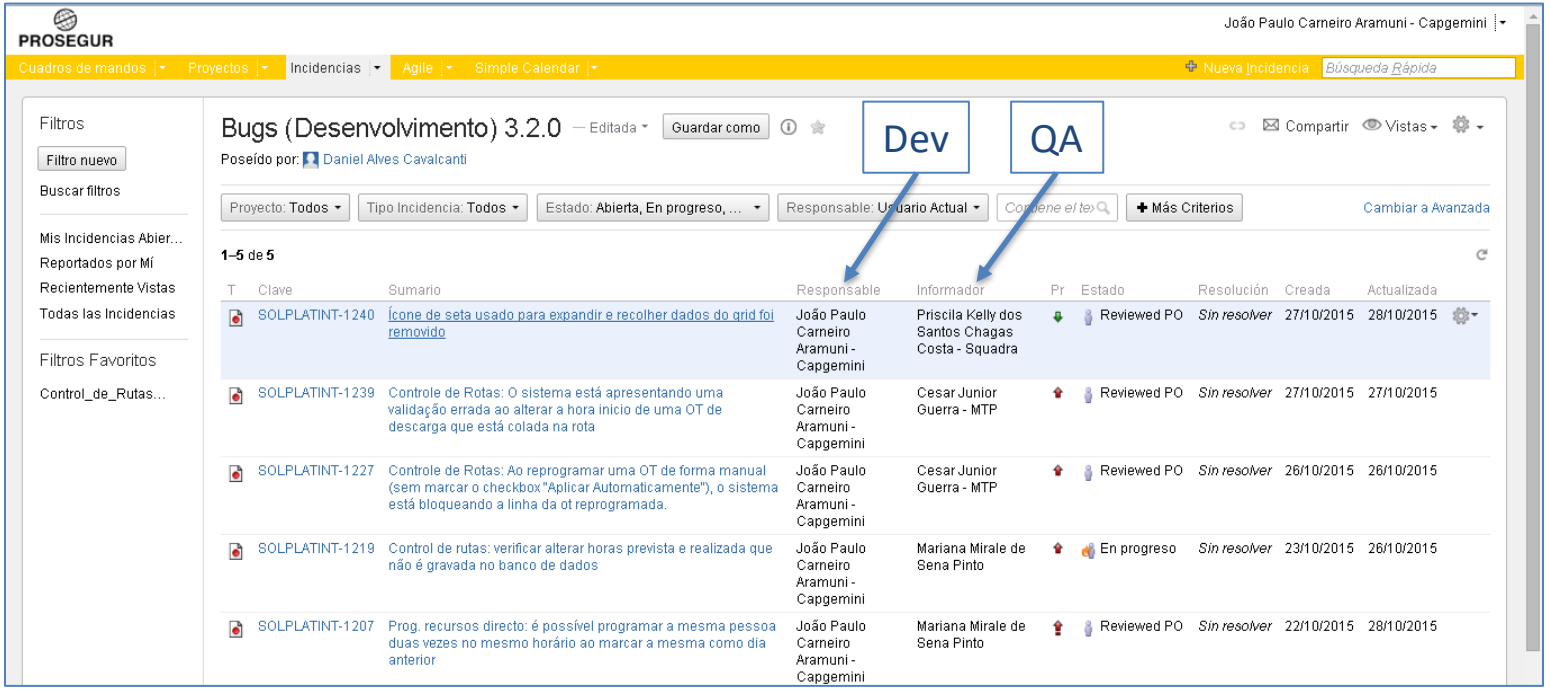
- **Importância da equipe de QA (Qualidade assegurada)**
  - **Melhoria contínua**: Contribuir para a evolução contínua dos processos de desenvolvimento, identificando oportunidades de melhoria.
  - **Entrega no prazo**: Reduzir a incidência de defeitos em produção, garantindo uma entrega pontual e suave.
  - **Custos reduzidos**: Minimizar custos associados à correção de defeitos após o lançamento.
  - **Colaboração eficaz**: Facilitar uma colaboração eficaz entre desenvolvedores, QA e outras partes interessadas.

Testes de resiliência dos serviços

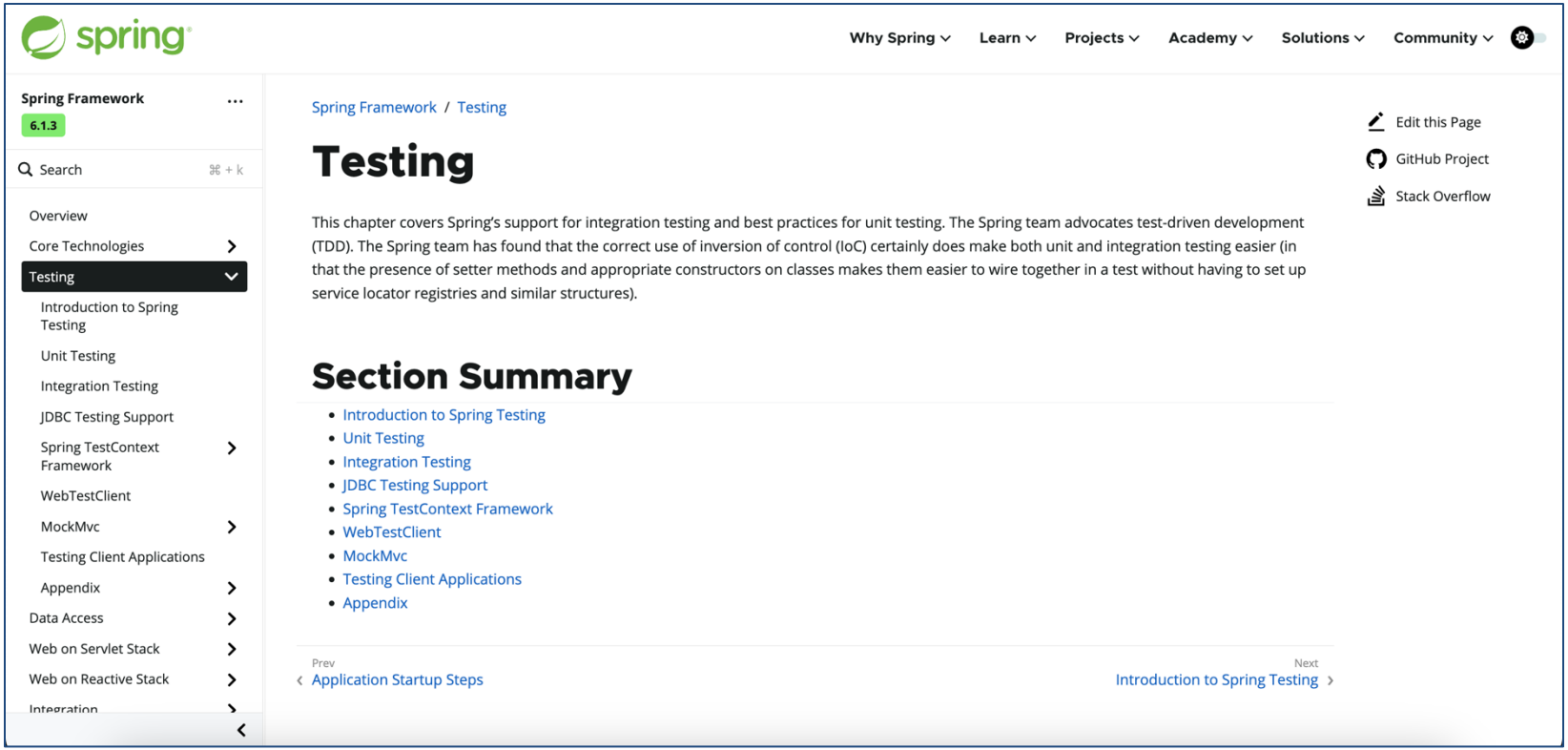
- **Equipe de QA (Qualidade assegurada)**
  - Antigamente, os profissionais do teste (analistas de QA, analistas de teste, testers, etc), praticamente não programavam.

  - Hoje em dia essa realidade mudou, forçando analistas de testes a se atualizarem e começarem a construir testes automatizados nas mais diversas linguagens de programação.

- Os bugs encontrados pelo time de teste são registrados em algum sistema, como por exemplo o **Jira**:
  - https://www.atlassian.com/br/software/jira

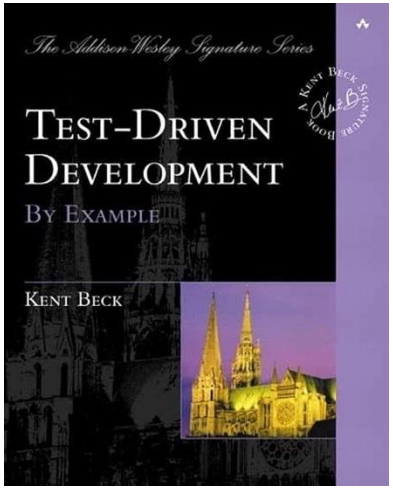


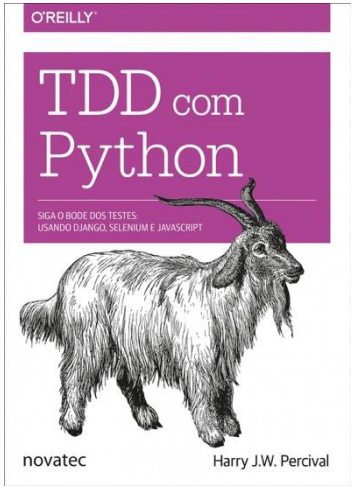
- O Spring possui bom suporte e documentação para testes:
  - https://docs.spring.io/spring-framework/reference/testing.html



Testes de resiliência dos serviços

- A equipe Spring defende o desenvolvimento orientado a testes (TDD), assim como diversas empresas de tecnologia pelo mundo.
- Sugestão de leitura:





Testes de resiliência dos serviços

- Para testar nossas aplicações Spring, utilizaremos basicamente três coisas:
  - A dependência: **spring-boot-starter-test**
    - https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-test
  - O framework **JUnit 5**, com seu módulo mais recente, **Jupiter**.
    - https://junit.org/junit5/docs/current/user-guide/#writing-tests
  - O framework de mocking: **Mockito**
    - https://site.mockito.org/

- **Spring Boot Starter Test**
  - O Spring Boot Starter Test é uma dependência do Spring Boot que fornece suporte para escrever testes em aplicações Spring Boot.

  - Este starter facilita a configuração do ambiente de teste e inclui várias bibliotecas e ferramentas comumente usadas para testes de aplicativos Spring Boot.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

- **JUnit 5 = Plataforma JUnit + JUnit Júpiter + JUnit Vintage**
  - A plataforma JUnit serve como base para o lançamento de estruturas de teste na JVM (Java Virtual Machine). Ela também define a API TestEngine para desenvolver um framework de teste que roda na plataforma.

  - JUnit Jupiter é a combinação do modelo de programação e do modelo de extensão para escrever testes e extensões no JUnit 5. O subprojeto Jupiter fornece um TestEngine para executar testes baseados em Jupiter na plataforma.



- **Mockito:** Estrutura de simulação ‘saborosa’ para testes unitários em Java
  - Mockito é um framework de mocking que permite que você escreva testes com uma API limpa e simples.
  - Mockito não dá ressaca porque os testes são muito legíveis e produzem erros de verificação limpos.
  - A enorme comunidade do StackOverflow elegeu o Mockito como o melhor framework de mocking para Java.



- Antes de diferenciarmos os tipos de teste, precisamos entender o conceito de **Mock** (ou Mockar – verbo carinhosamente inventado pelos brasileiros).
- "Mockar objetos” refere-se ao processo de criar objetos simulados (mocks) que imitam o comportamento de objetos reais em um ambiente controlado. Como se fosse um **dublê**.







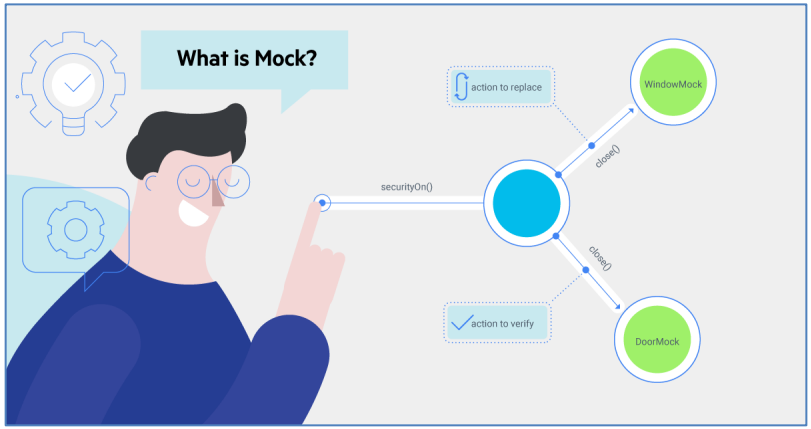
Testes de resiliência dos serviços

- Esses objetos simulados são usados para isolar a unidade de código que está sendo testada, garantindo que o teste se concentre apenas na lógica específica dessa unidade e não em suas dependências externas.

- Ao mockar objetos, você pode simular o comportamento de componentes externos, como bancos de dados, serviços web, APIs ou outros colaboradores, sem depender da implementação real desses componentes durante os testes.

- Isso é útil para criar testes isolados e consistentes, onde você pode controlar as entradas e prever as saídas, facilitando a identificação e correção de problemas.

- **Mockar** é uma prática comum em testes unitários, especialmente quando se trabalha com **TDD** (Desenvolvimento Orientado a Testes) ou em situações em que se deseja isolar partes específicas do código para garantir testes mais precisos e eficientes.
- Leitura sugerida:
  - https://martinfowler.com/articles/mocksArentStubs.html



Testes de resiliência dos serviços

- **Tipos de teste**
  - **Teste Unitário**:
    - Teste focado em verificar a menor unidade de código, como uma função ou método, isoladamente. É utilizado para garantir que cada unidade funcione conforme esperado.

    - **Objetivo**: Certificar-se de que cada componente individual do software opera corretamente.

Testes de resiliência dos serviços

- **Tipos de teste**
  - **Teste de Integração**:
    - Verifica a interação entre diferentes unidades ou módulos de código. O objetivo é garantir que as partes do sistema funcionem bem juntas quando integradas.

    - **Objetivo**: Identificar problemas que podem surgir devido à colaboração entre componentes.

Testes de resiliência dos serviços

- **Tipos de teste**
  - **Teste Funcional:**
    - Avalia se o sistema atende às especificações e requisitos funcionais. É conduzido do ponto de vista do usuário, verificando se as funcionalidades do software estão conforme o esperado.

    - **Objetivo**: Garantir que o software atenda às necessidades e expectativas dos usuários.

Testes de resiliência dos serviços

- **Tipos de teste**
  - **Teste de Regressão:**
    - Após uma alteração no código, este teste verifica se as funcionalidades existentes ainda estão operando corretamente, garantindo que novas implementações não quebrem funcionalidades existentes.

    - **Objetivo**: Prevenir regressões no software após modificações.

Testes de resiliência dos serviços

- **Tipos de teste**
  - **Teste de Desempenho:**
    - Descrição: Avalia o desempenho do sistema em condições específicas, como carga máxima, volume de dados ou concorrência. Identifica gargalos e otimizações necessárias.

    - **Objetivo**: Garantir que o sistema atenda aos requisitos de desempenho.

Testes de resiliência dos serviços

- **Tipos de teste**
  - **Teste de Carga:**
    - Avalia o comportamento do sistema sob carga extrema. Testa a capacidade do sistema de lidar com um grande número de usuários simultaneamente.

    - **Objetivo**: Identificar limitações relacionadas à capacidade e escalabilidade.

Testes de resiliência dos serviços

- **Tipos de teste**
  - **Teste de Estresse:**
    - Avalia como o sistema se comporta em condições extremas ou além das especificações normais. Visa identificar o ponto de falha ou comportamento inesperado sob estresse significativo.

    - **Objetivo**: Avaliar a robustez e estabilidade do sistema em situações críticas.

Testes de resiliência dos serviços

- **Tipos de teste**
  - **Teste de Usabilidade:**
    - Avalia a facilidade de uso e a experiência do usuário. Envolve testar a interface do usuário para garantir que seja intuitiva e amigável.

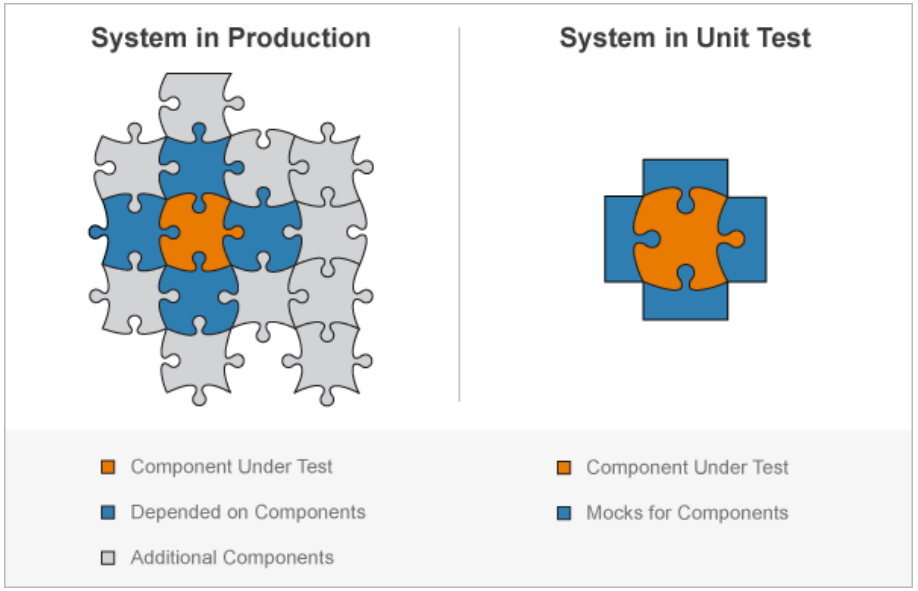
    - **Objetivo**: Certificar-se de que o software é acessível e compreensível para os usuários finais.

Sumário

- Testes de resiliência dos serviços
  - **Teste unitário**
  - Teste de desempenho
  - Teste de carga

Testes de resiliência dos serviços

- **Teste unitário**
  - Como Devs, precisamos, no mínimo, nos preocupar em construir testes unitários.
  - Estes que precisamos garantir que estejam presentes em todas as situações.

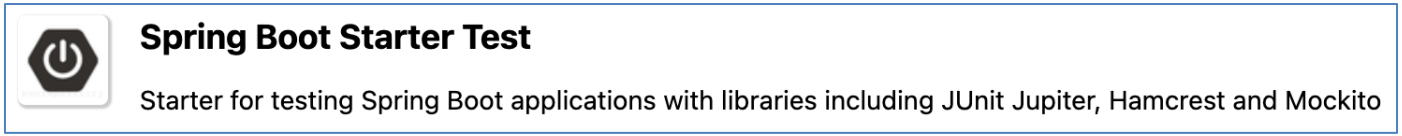


Testes de resiliência dos serviços

- **Teste unitário**
  - Vamos complementar o projeto com banco de dados da aula anterior e testar os cinco métodos que construímos:
    - obterTodos()
    - obterPorId()
    - inserir()
    - atualizar()
    - excluir()

Testes de resiliência dos serviços

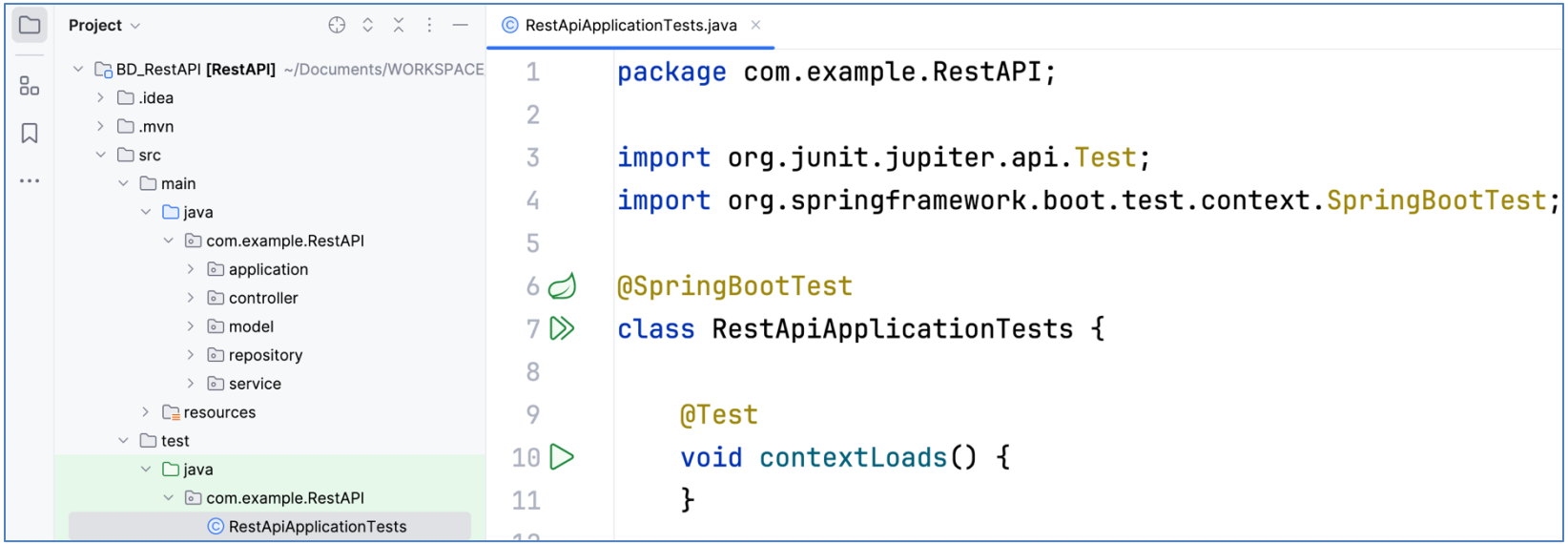
- **Teste unitário**
  - Como já incluímos a dependência spring-boot-starter-test, não precisamos incluir o JUnit Jupiter nem o Mockito, pois ambos já fazem parte deste pacote.




```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

Arquivo pom.xml

- **Teste unitário**
  - O Spring Boot já gerou automaticamente uma pasta test com um arquivo de test padrão no momento da criação do projeto.



Testes de resiliência dos serviços

- **Teste unitário**
  - Criaremos cinco testes unitários usando o Spring Boot Starter Test, JUnit 5 e Mockito.

  - Porém, antes disso, precisamos instalar o Apache **Maven**.
    - https://maven.apache.org/install.html

Testes de resiliência dos serviços

- **Apache Maven**
  - O Apache Maven é uma ferramenta de automação de construção e gerenciamento de projetos amplamente utilizada na construção de software Java.
  - Ele simplifica o processo de compilação, empacotamento e distribuição de um projeto Java, fornecendo uma estrutura padrão e uma maneira consistente de gerenciar dependências.



Testes de resiliência dos serviços

- Para instalar o Maven:
  - No Linux:
    - sudo apt update
    - sudo apt install maven
  - No MacOS:
    - brew install maven

- Depois de instalado, os principais comandos que iremos utilizar são:
  - mvn clean e **mvn test**

Testes de resiliência dos serviços

- Principais recursos do Maven:
  - **Gerenciamento de dependências**:
    - O Maven facilita a gestão das dependências do projeto.
    - Você pode especificar as bibliotecas e frameworks necessários no arquivo de configuração **pom.xml**, e o Maven se encarregará de baixar e gerenciar essas dependências.

Testes de resiliência dos serviços

- Principais recursos do Maven:
  - **Convenção sobre configuração**:
    - O Maven segue uma abordagem de "convenção sobre configuração".
    - Isso significa que muitas configurações padrão são assumidas, e você só precisa especificar o que é diferente do padrão.

Testes de resiliência dos serviços

- Principais recursos do Maven:
  - **Ciclo de vida padrão**:
    - O Maven define um ciclo de vida padrão para a construção de projetos. Isso inclui fases como compile, test, package, install e deploy.
    - Cada fase executa um conjunto específico de metas (goals) associadas a tarefas específicas.

Testes de resiliência dos serviços

- Principais recursos do Maven:
  - **Empacotamento uniforme**:
    - O Maven permite que você empacote seu projeto de maneira consistente.
    - Você pode criar JARs, WARs, EARs e outros artefatos de maneira padronizada.

Testes de resiliência dos serviços

- Principais recursos do Maven:
  - **Plugins**:
    - O Maven é altamente extensível por meio de plugins.
    - Diversos plugins estão disponíveis para realizar uma variedade de tarefas, desde a execução de testes até a geração de documentação.

Testes de resiliência dos serviços

- Principais recursos do Maven:
  - **Repositórios Maven**:
    - O Maven utiliza repositórios para armazenar e recuperar artefatos.
    - Existem repositórios públicos e privados, e o Maven ajuda a baixar automaticamente as dependências do projeto a partir desses repositórios.

Testes de resiliência dos serviços

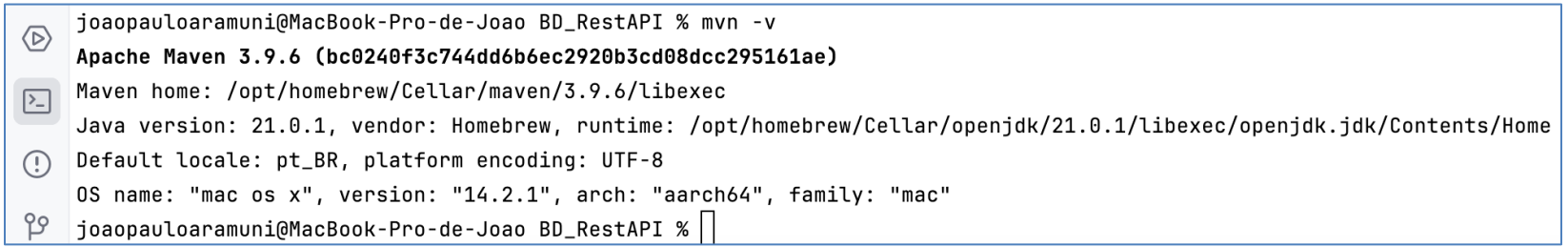
- Principais recursos do Maven:
  - **Multi-módulo**:
    - O Maven suporta projetos com vários módulos, facilitando a gestão de projetos complexos com várias partes inter-relacionadas.

    - O conceito de projeto multi-módulo no Maven refere-se à estrutura em que um único projeto é dividido em vários módulos independentes.

Testes de resiliência dos serviços

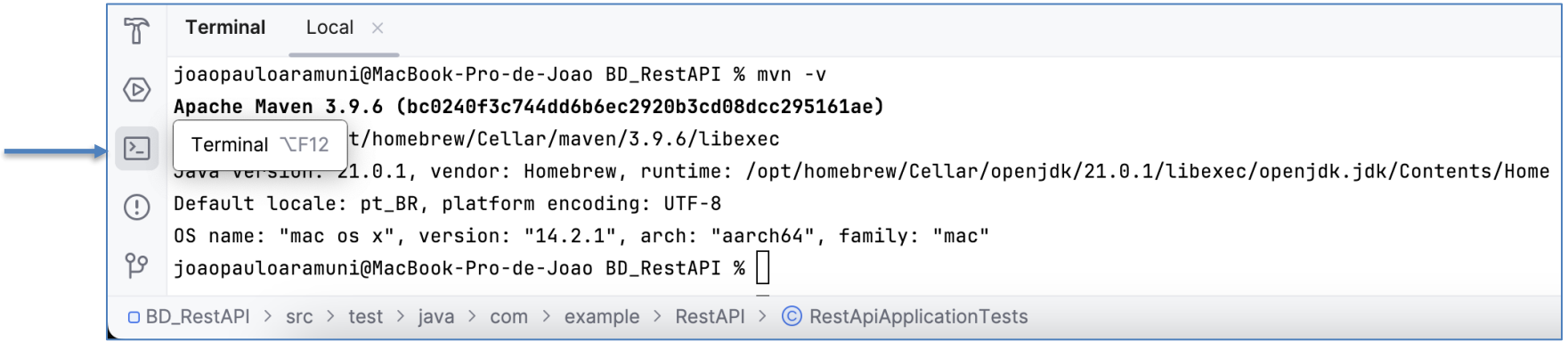
- Ao usar o Maven, os desenvolvedores podem se concentrar mais no desenvolvimento do código-fonte e menos na configuração e na gestão de dependências, tornando o processo de construção de software mais eficiente e padronizado.

- Para testar sua instalação rode: **mvn -v**

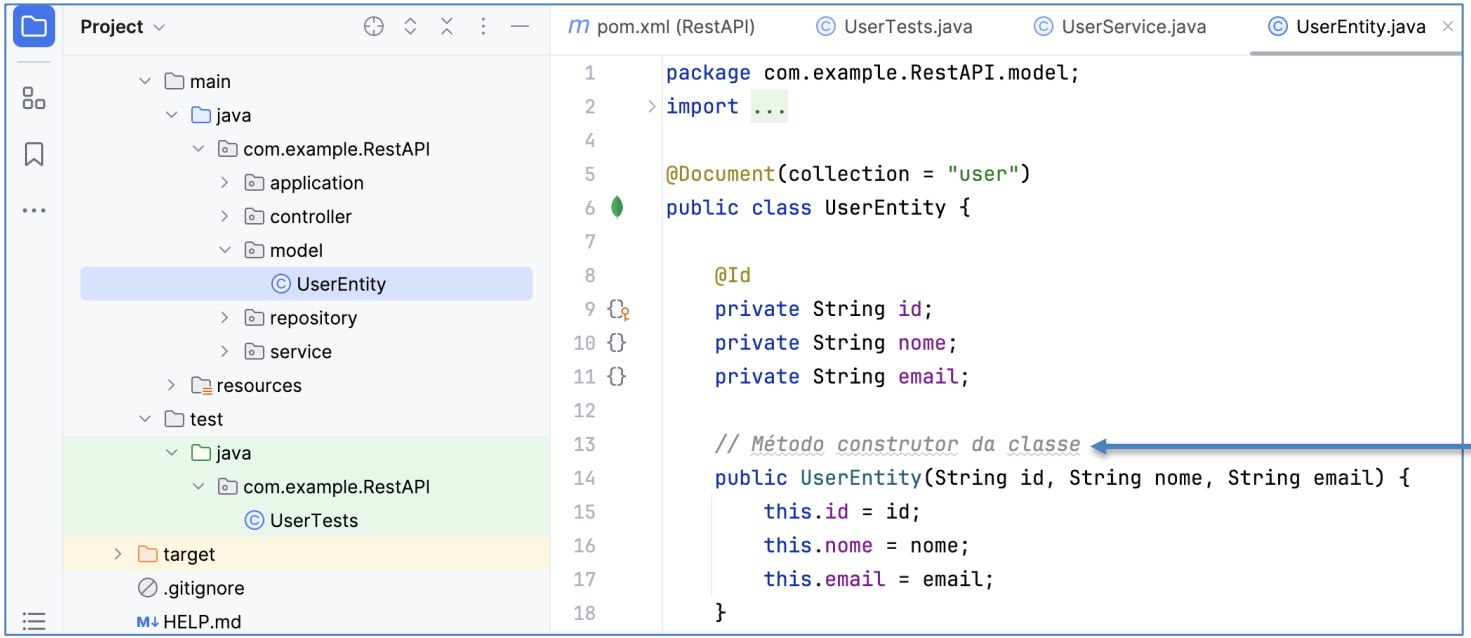


Testes de resiliência dos serviços

- Lembrando que esta instalação pode ser feita no terminal do próprio **IntelliJ** IDEA, na barra inferior:



- **Teste unitário**
  - Antes de partirmos para a classe de testes, precisamos inserir um método construtor em nossa entidade UserEntity:

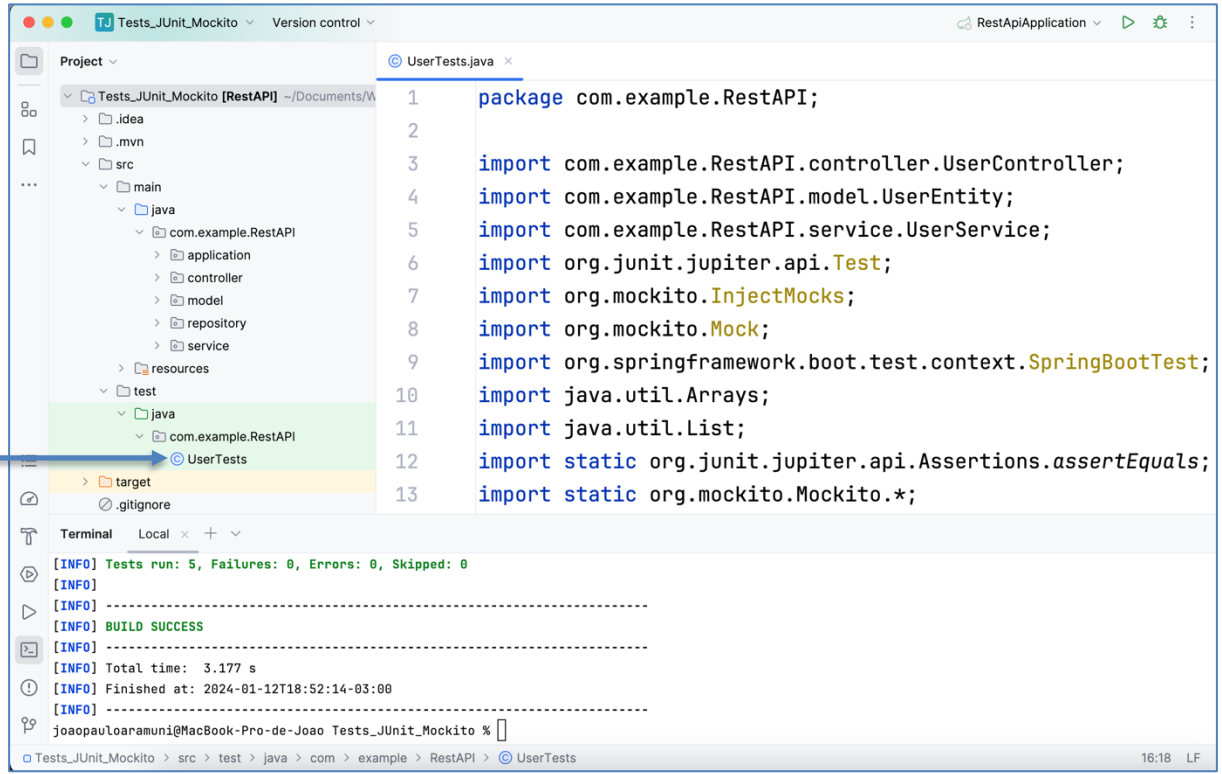


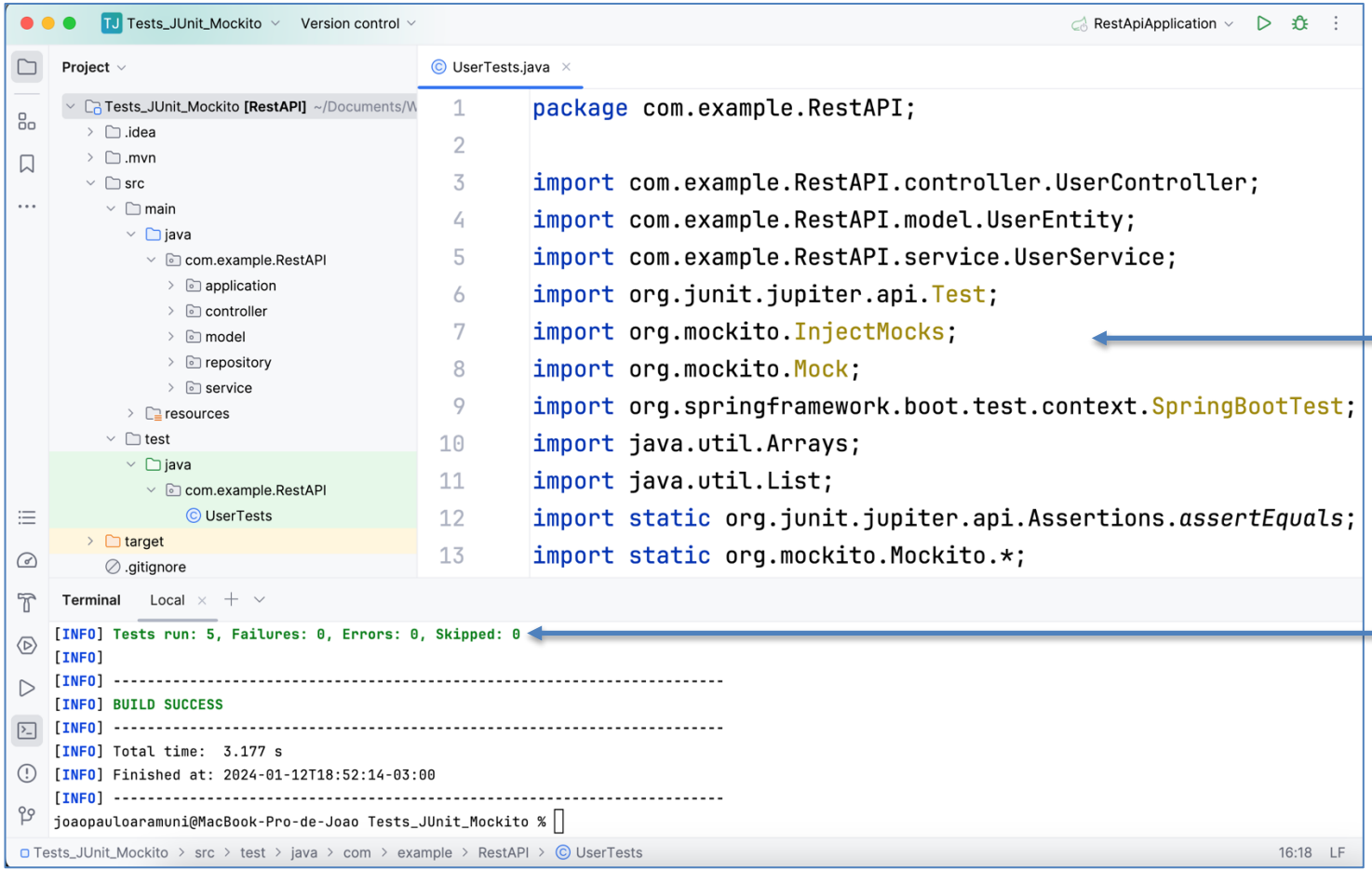
- **Teste unitário**
  - Esse método construtor possibilitará a instanciação de objetos da classe **UserEntity** em nossa classe de teste (**UserTests**).
    - Ao adicionar um construtor personalizado, garantimos que podemos criar objetos UserEntity com dados controlados durante a execução dos testes.

```
@Test
void testObterTodos() {
    // Simular dados de usuário
    List<UserEntity> userList = Arrays.asList(
            new UserEntity( id: "1",  nome: "User1",  email: "user1@example.com"),
            new UserEntity( id: "2",  nome: "User2",  email: "user2@example.com")
    );
```

- **Teste unitário**
  - Vamos criar então a nossa primeira classe de testes, a **UserTests**.

UserTests

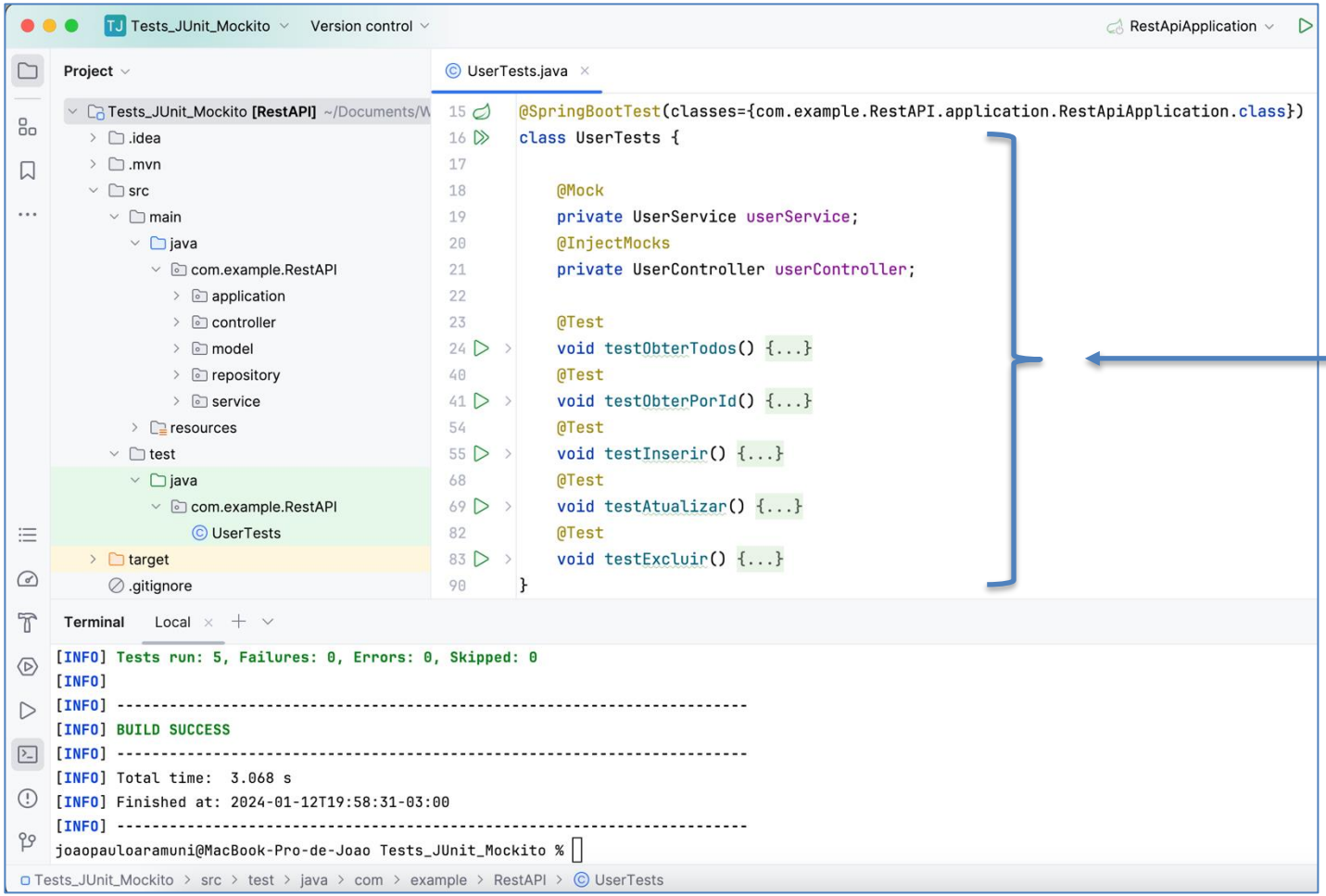




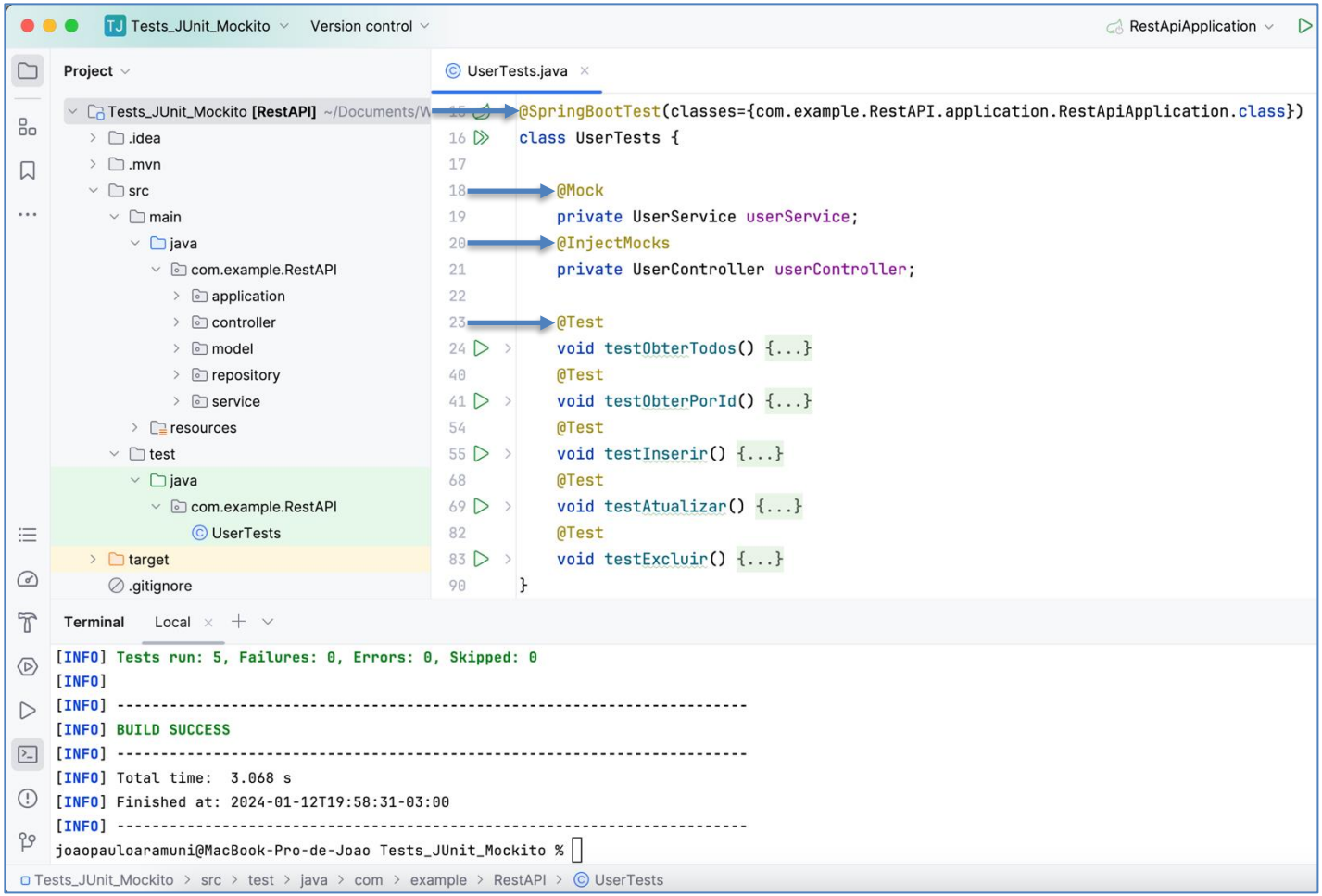
Aqui estão todos os **imports** que precisamos fazer

Aqui no terminal veremos os resultados dos testes que estamos construindo.

Usaremos para isso o comando **mvn test**



Essa é a estrutura base da nossa classe de testes.



@SpringBootTest
@Mock
@InjectMocks
@Test

Vejamos primeiro o que cada uma destas **annotations** fazem

Testes de resiliência dos serviços

- @SpringBootTest:
  - Esta anotação é usada em testes de integração no Spring Boot.
  - Indica que o teste deve carregar a aplicação completa, incluindo todas as configurações, e criar um contexto de aplicação para execução dos testes.
  - Essa annotation é útil para testes que envolvem várias camadas da aplicação, como testes de integração entre componentes.

  - `@SpringBootTest(classes={com.example.RestAPI.application.RestApiApplication.class})`

Testes de resiliência dos serviços

- @Mock:
  - A anotação @Mock faz parte do framework Mockito e é usada para criar um objeto simulado, conhecido como "mock".
  - Um mock é uma implementação simulada de uma classe ou interface que pode ter comportamentos específicos configurados para atender às necessidades de um teste.
  - Esses objetos simulados permitem isolar o componente em teste de suas dependências, concentrando-se apenas no comportamento específico do componente.

  - ```
    @Mock
    private UserService userService;
    ```
    ```

Testes de resiliência dos serviços

- @InjectMocks:
  - Também proveniente do Mockito, a anotação @InjectMocks é usada para injetar automaticamente os mocks (criados com @Mock) em uma instância da classe que está sendo testada.
  - Simplifica a configuração de testes, permitindo que o Mockito cuide da injeção de dependências automaticamente.

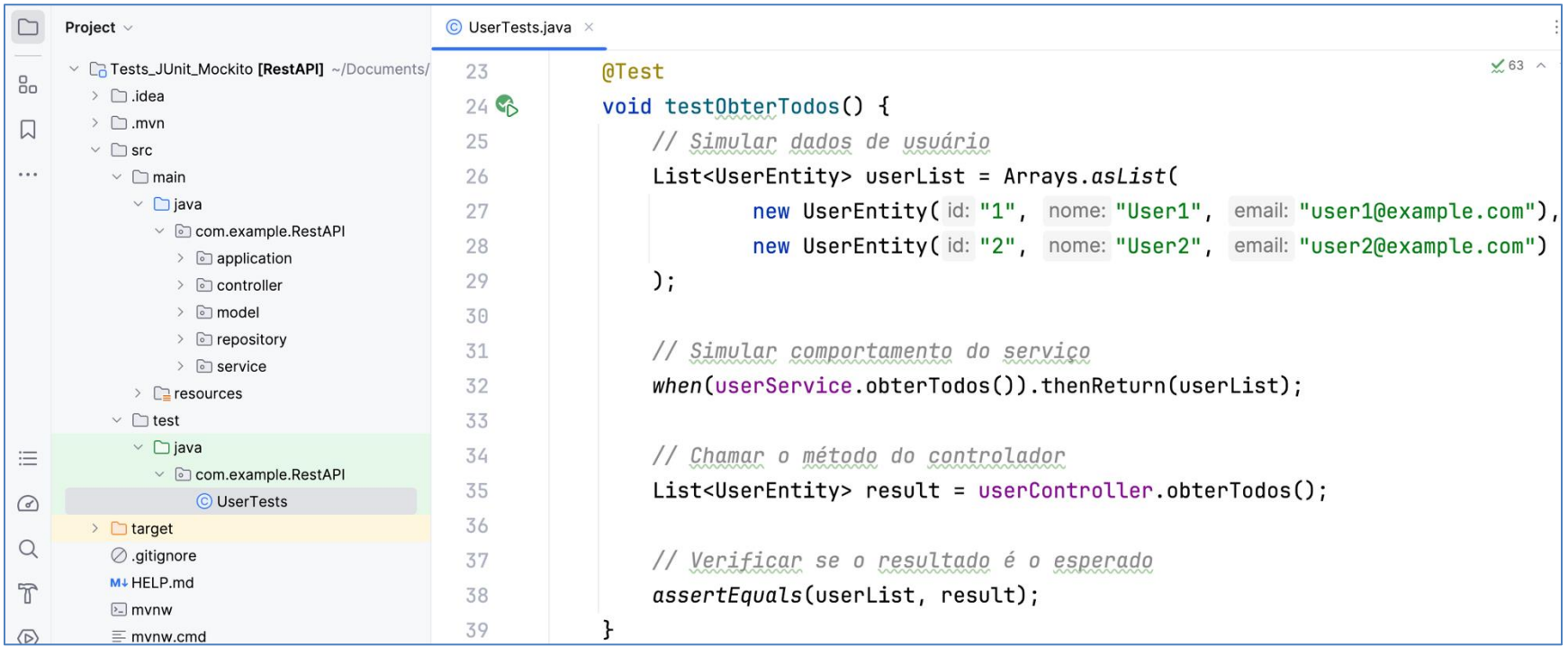
  - ```
    @InjectMocks
    private UserController userController;
    ```
    ```

Testes de resiliência dos serviços

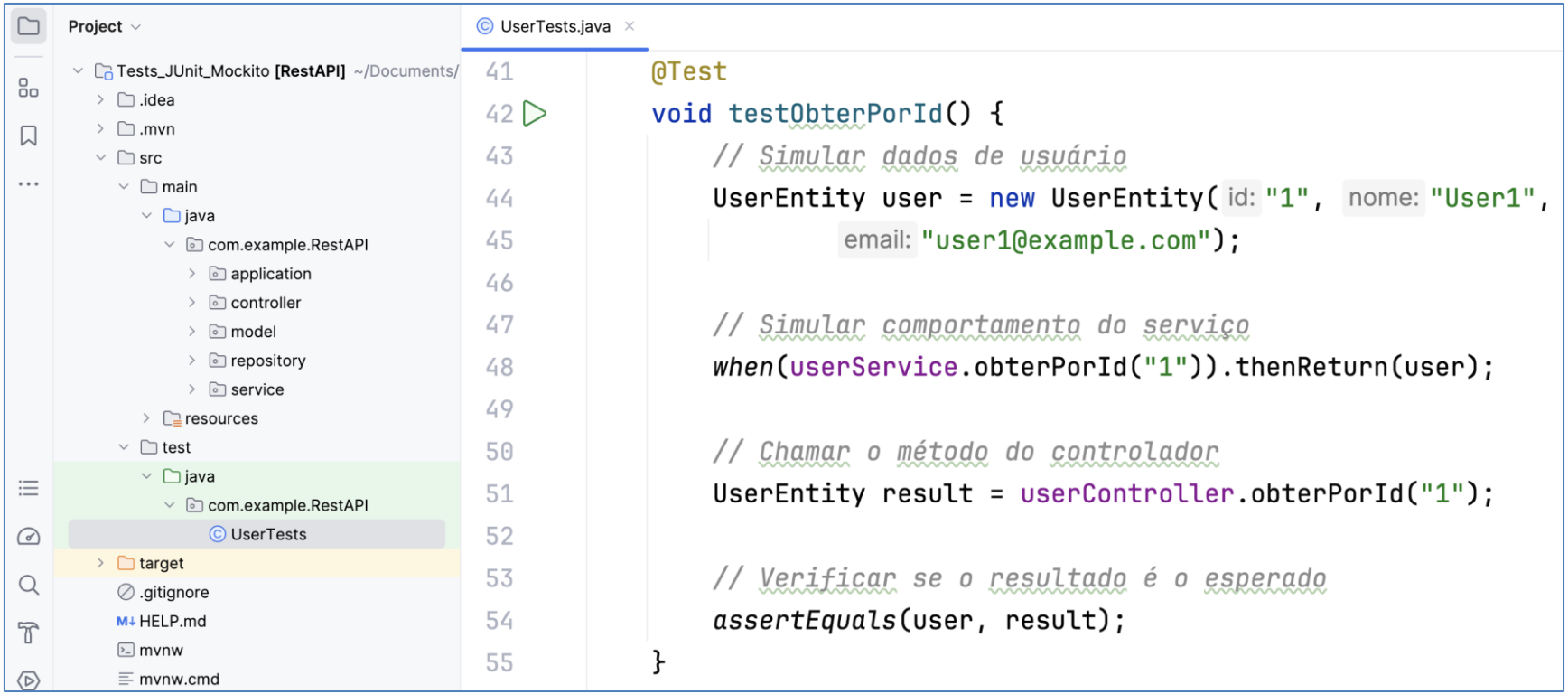
- @Test:
  - A anotação @Test faz parte do JUnit, um framework de teste amplamente utilizado em Java.
  - Indica que um método é um método de teste.
  - Os métodos marcados com @Test são executados durante a execução do teste e contêm as verificações e as lógicas para testar o comportamento esperado do código.

  - ```
    @Test
    void testObterTodos()
    ```
    ```

- Vejamos agora os cinco testes unitários, um a um.
  - void testObterTodos()



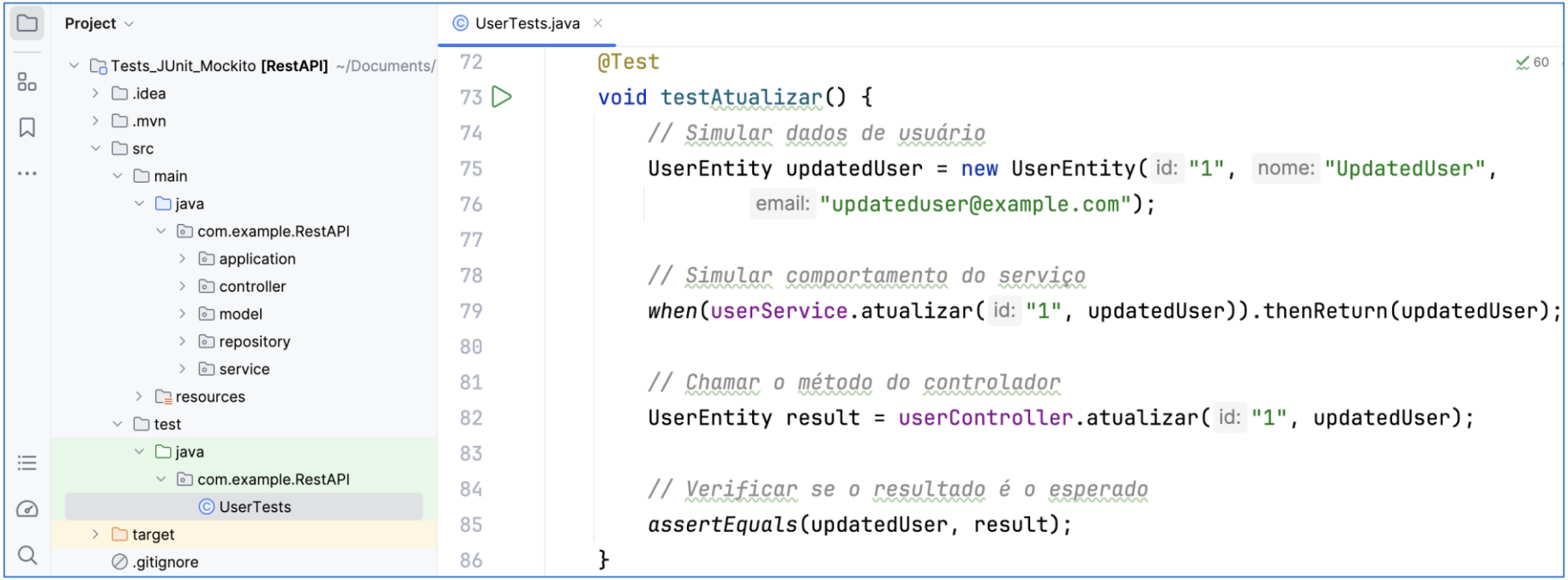
- Vejamos agora os cinco testes unitários, um a um.
  - void testObterPorId()



- Vejamos agora os cinco testes unitários, um a um.
  - void testInserir()



- Vejamos agora os cinco testes unitários, um a um.
  - void testAtualizar()



- Vejamos agora os cinco testes unitários, um a um.
  - void testExcluir()



- //Simular **dados de usuário**:
  - Aqui estão nossos dados simulados, ou dublês.
  - Estes dados **não** serão gravados no banco de dados e servem apenas para teste.
  - O testExcluir() não precisa simular dados.

```
@Test
void testObterTodos() {
    // Simular dados de usuário
    List<UserEntity> userList = Arrays.asList(
            new UserEntity( id: "1",  nome: "User1",  email: "user1@example.com"),
            new UserEntity( id: "2",  nome: "User2",  email: "user2@example.com")
    );
```

```
@Test
void testInserir() {
    // Simular dados de usuário
    UserEntity newUser = new UserEntity( id: "1",  nome: "User1",
            email: "user1@example.com");
```

```
@Test
void testObterPorId() {
    // Simular dados de usuário
    UserEntity user = new UserEntity( id: "1",  nome: "User1",
            email: "user1@example.com");
```

```
@Test
void testAtualizar() {
    // Simular dados de usuário
    UserEntity updatedUser = new UserEntity( id: "1",  nome: "UpdatedUser",
            email: "updateduser@example.com");
```

- //Simular comportamento do **serviço**:
  - Método **when** do Mockito.
  - O testExcluir() não retorna dados.

```
// Simular comportamento do serviço
when(userService.obterTodos()).thenReturn(userList);
```

```
// Simular comportamento do serviço
when(userService.obterPorId("1")).thenReturn(user);
```

```
// Simular comportamento do serviço
when(userService.inserir(newUser)).thenReturn(newUser);
```

```
// Simular comportamento do serviço
when(userService.atualizar( id: "1", updatedUser)).thenReturn(updatedUser);
```

```
// Simular comportamento do serviço
doNothing().when(userService).excluir( id: "1");
```

Testes de resiliência dos serviços

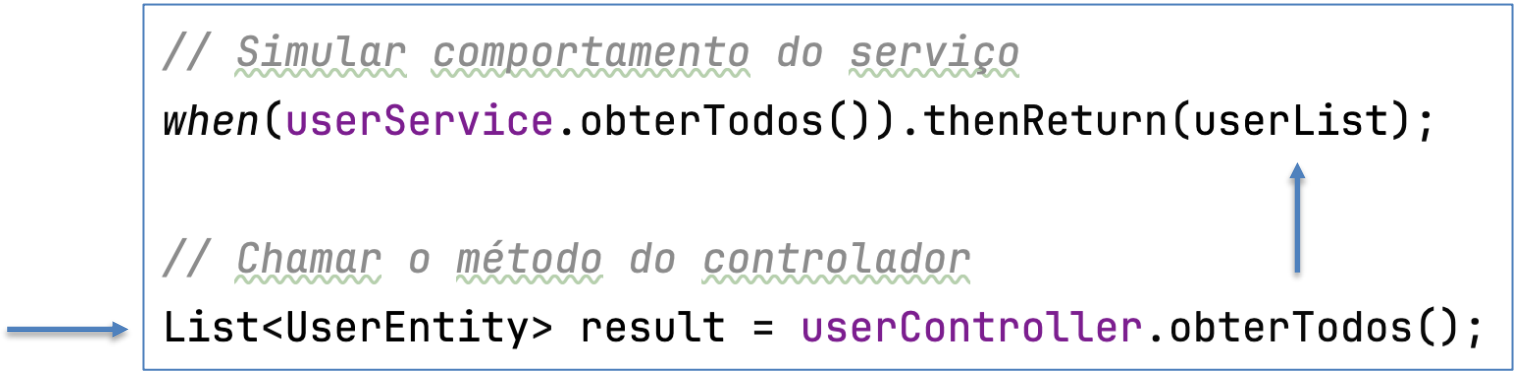
- Método **when()** do Mockito:
  - O método when do Mockito é usado para configurar o comportamento esperado de um método de um objeto mockado.
  - Ele permite que você especifique o que deve acontecer quando um determinado método do mock for chamado durante a execução do teste.
  - ***when****(mockObjeto.metodo()).****thenReturn****(valorDeRetorno);*

Testes de resiliência dos serviços

- Método **when()** do Mockito:
  - ***when****(mockObjeto.metodo()).****thenReturn****(valorDeRetorno);*
    - **mockObjeto**: O objeto que foi criado como mock (mockado).
    - **metodo()**: O método específico do objeto mockado para o qual você deseja configurar o comportamento.
    - **thenReturn**(valorDeRetorno): Define o que o método deve retornar quando for chamado.

Testes de resiliência dos serviços

- Método **when()** do Mockito:
  - Estamos configurando o comportamento esperado do serviço antes de efetuar a chamada real do método do controlador.
  - Estamos definindo as **expectativas** antes de ocorrerem as interações reais.



- //Chamar o método do controlador:
  - O próximo passo é a chamada real ao método do controller.

```
// Chamar o método do controlador
List<UserEntity> result = userController.obterTodos();
```

```
// Chamar o método do controlador
UserEntity result = userController.obterPorId("1");
```

```
// Chamar o método do controlador
UserEntity result = userController.inserir(newUser);
```

```
// Chamar o método do controlador
UserEntity result = userController.atualizar( id: "1", updatedUser);
```

```
// Chamar o método do controlador
userController.excluir( id: "1");
```

- Vejamos a simulação do comportamento do serviço juntamente com a chamada ao método do controlador.

testObterTodos()

```
// Simular comportamento do serviço
when(userService.obterTodos()).thenReturn(userList);

// Chamar o método do controlador
List<UserEntity> result = userController.obterTodos();
```

testInserir()

```
// Simular comportamento do serviço
when(userService.inserir(newUser)).thenReturn(newUser);

// Chamar o método do controlador
UserEntity result = userController.inserir(newUser);
```

testObterId()

```
// Simular comportamento do serviço
when(userService.obterPorId("1")).thenReturn(user);

// Chamar o método do controlador
UserEntity result = userController.obterPorId("1");
```

testExcluir()

```
// Simular comportamento do serviço
doNothing().when(userService).excluir( id: "1");

// Chamar o método do controlador
userController.excluir( id: "1");
```

testAtualizar()

```
// Simular comportamento do serviço
when(userService.atualizar( id: "1", updatedUser)).thenReturn(updatedUser);

// Chamar o método do controlador
UserEntity result = userController.atualizar( id: "1", updatedUser);
```

- // Verificar se o resultado é o esperado
  - Por último, precisamos verificar se o resultado é o esperado.
  - No caso do testExcluir(), verificar se o método de serviço foi chamado.

```
// Verificar se o resultado é o esperado
assertEquals(userList, result);
```

testObterTodos()

```
// Verificar se o resultado é o esperado
assertEquals(user, result);
```

testObterPorId()

```
// Verificar se o resultado é o esperado
assertEquals(newUser, result);
```

testInserir()

```
// Verificar se o resultado é o esperado
assertEquals(updatedUser, result);
```

testAtualizar()

```
// Verificar se o método de serviço foi chamado
verify(userService, times( wantedNumberOfInvocations: 1)).excluir( id: "1");
```

testExcluir()

Testes de resiliência dos serviços

- Método **assertEquals()** do JUnit:
  - O método assertEquals é usado para verificar se dois objetos são iguais.
  - No testAtualizar(), por exemplo, estamos comparando o objeto updatedUser (que seria o resultado esperado após uma operação de atualização) com o objeto result (que é o resultado real obtido após chamar um método no controlador).

```
// Verificar se o resultado é o esperado
assertEquals(updatedUser, result);
```

testAtualizar()

Testes de resiliência dos serviços

- Método **assertEquals()** do JUnit:
  - Em contexto de teste, uma **asserção** (ou afirmação) é uma expressão que verifica se uma condição específica é verdadeira.
  - Em outras palavras, é uma declaração que você faz para verificar se algo esperado está acontecendo durante a execução do teste.
  - Se a condição não for atendida, a asserção falhará, indicando que algo está errado.

Testes de resiliência dos serviços

- Método **verify()** do Mockito:
  - O método verify é usado para verificar se um determinado método em um mock foi chamado com os argumentos esperados e na quantidade esperada.

  - No testExcluir(), estamos verificando se o método excluir("1") do mock userService foi chamado exatamente uma vez (times(1)) durante o teste.

```
// Verificar se o método de serviço foi chamado
verify(userService, times( wantedNumberOfInvocations: 1)).excluir( id: "1");
```

testExcluir()

Testes de resiliência dos serviços

- Método **verify()** do Mockito:
  - Essa verificação é útil para garantir que certos métodos em seus mocks sejam invocados corretamente durante a execução do código testado.

  - Se a verificação falhar, o teste também falhará, indicando que o método esperado não foi chamado ou que foi chamado com argumentos diferentes.

```
// Verificar se o método de serviço foi chamado
verify(userService, times( wantedNumberOfInvocations: 1)).excluir( id: "1");
```

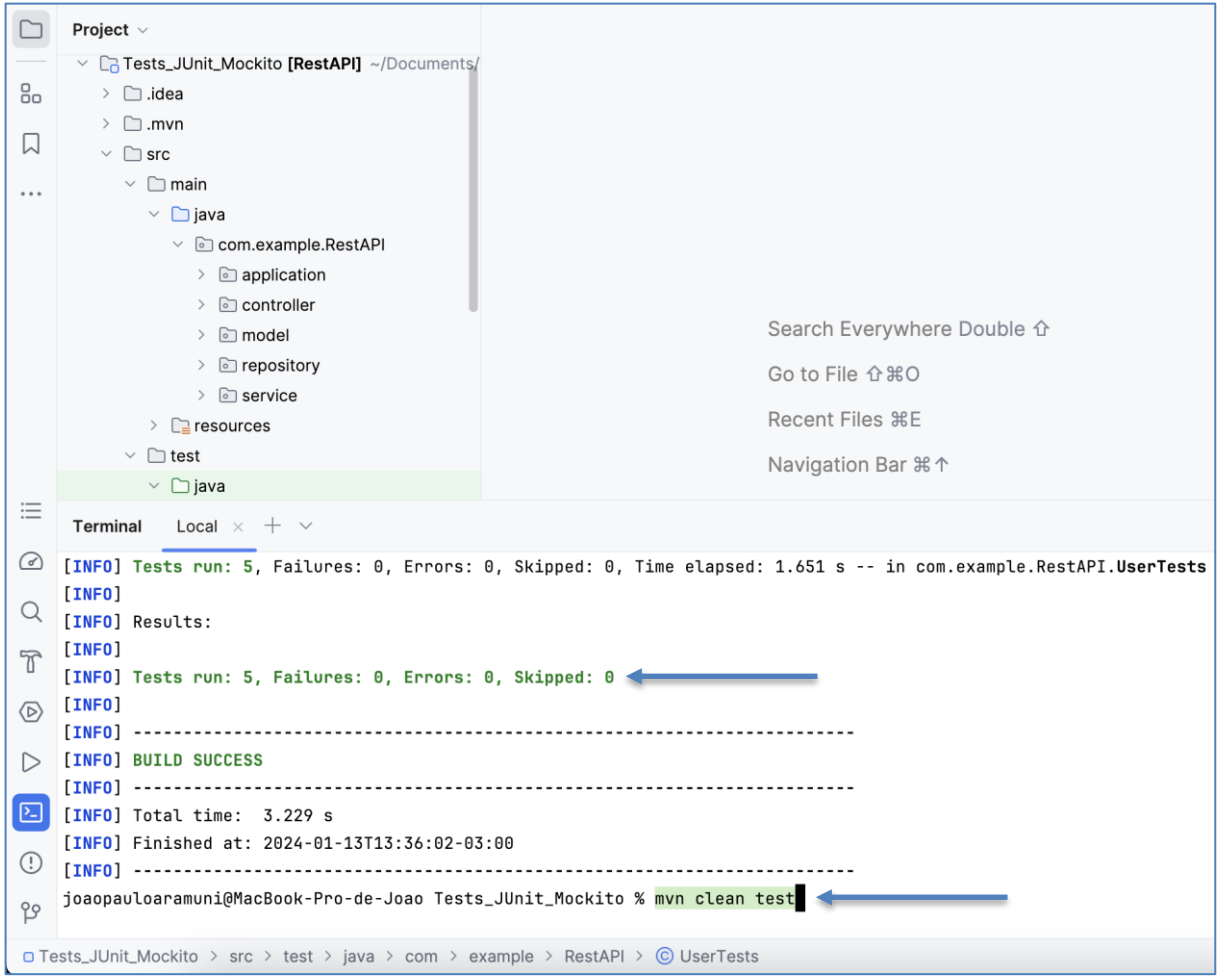
testExcluir()

Testes de resiliência dos serviços

- **mvn clean**:
  - O comando clean é utilizado para limpar (ou remover) os artefatos gerados durante o processo de compilação anterior.
  - Ele exclui o diretório target (ou outro diretório de saída configurado) e remove quaisquer arquivos compilados, classes, JARs e outros artefatos gerados durante o processo de compilação.
  - É útil quando você deseja garantir que está começando uma compilação "limpa" do zero, sem artefatos antigos.

Testes de resiliência dos serviços

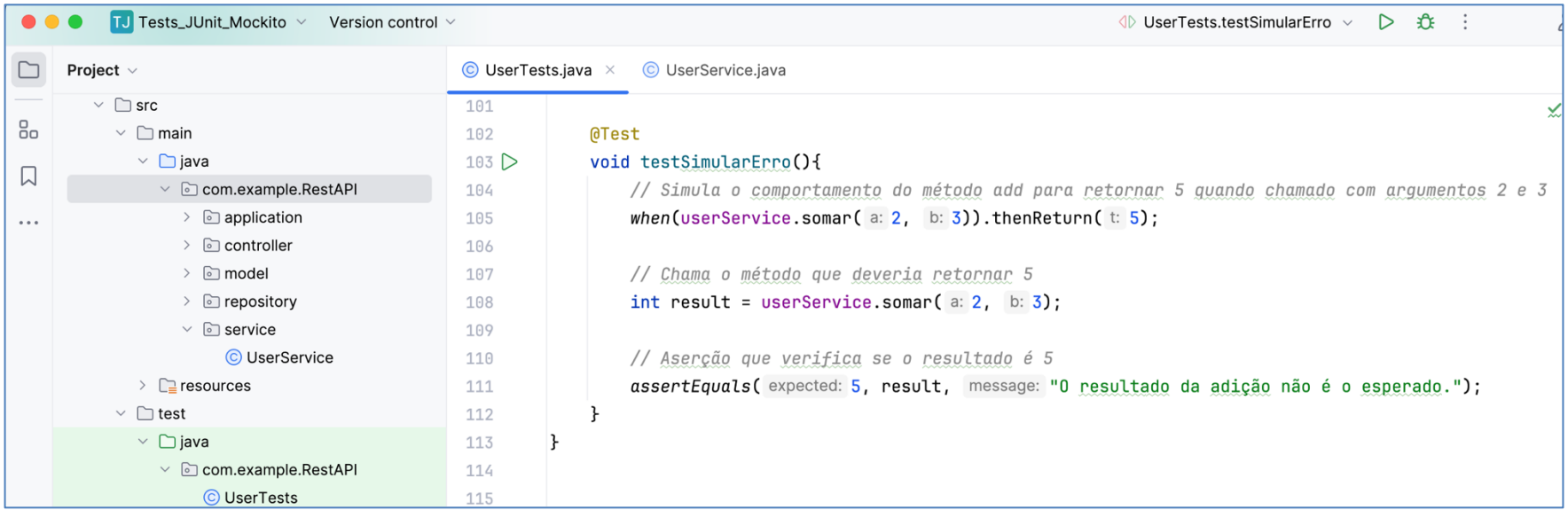
- **mvn test**:
  - O comando test é usado para executar os testes unitários definidos em seu projeto.
  - Ele compila o código-fonte, compila os testes, e então executa os testes unitários definidos em seu projeto.
  - Seu objetivo principal é garantir que o código está funcionando conforme esperado, identificando possíveis falhas ou problemas durante a execução dos testes.



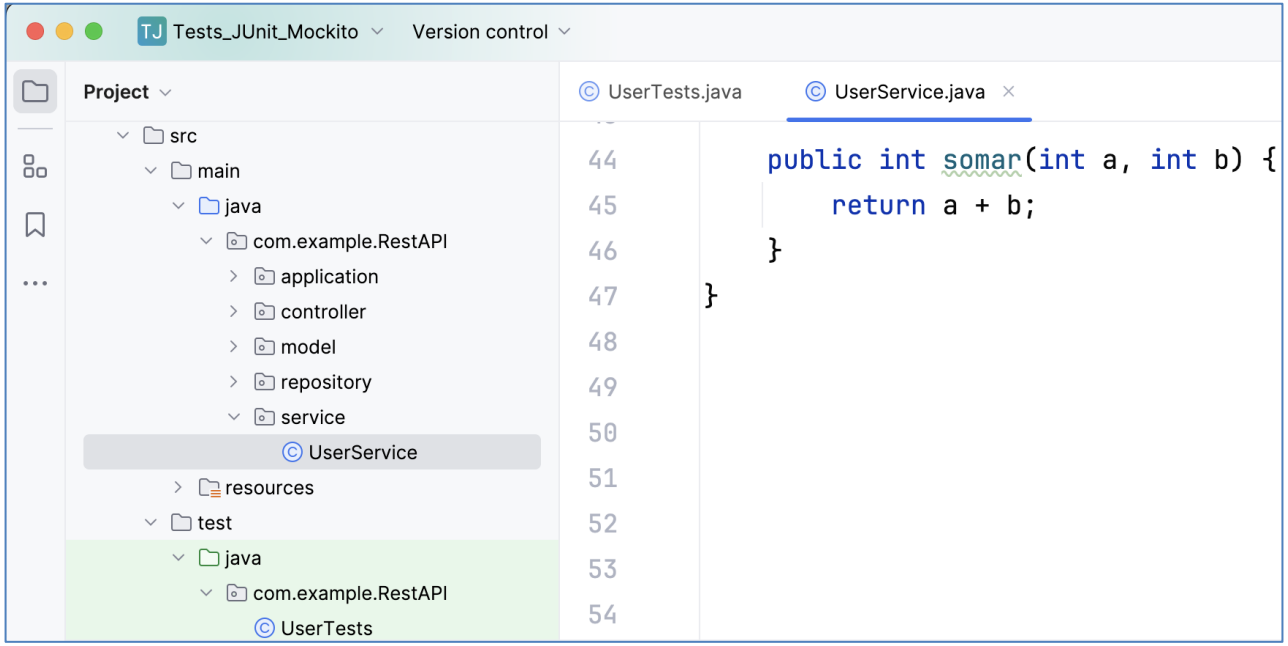
mvn clean test

Teste você mesmo no terminal do IntelliJ IDEA

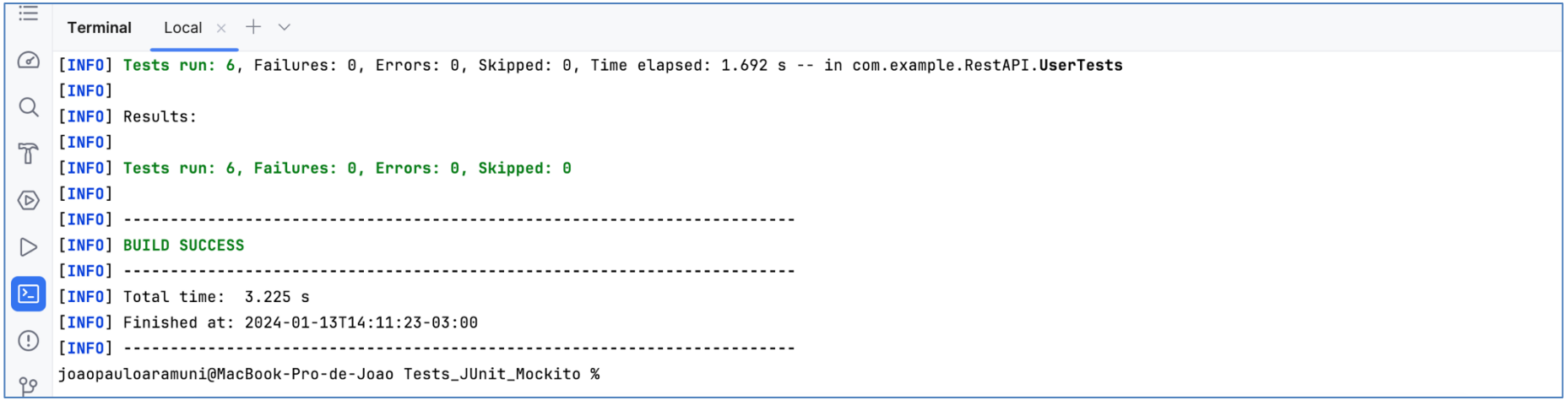
- **Forçando um erro**:
  - Vamos agora simular um **erro** durante os nossos testes.
  - testSimularErro()



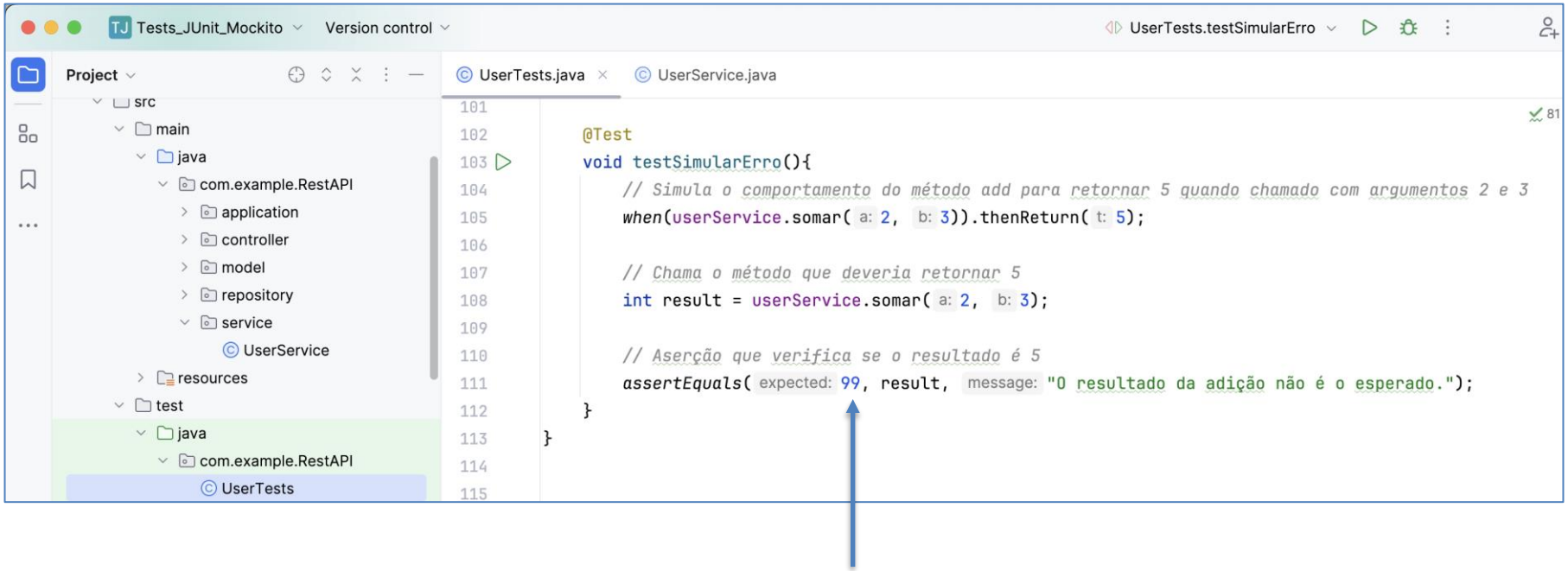
- **Forçando um erro**:
  - Vamos agora simular um **erro** durante os nossos testes.
  - testSimularErro()

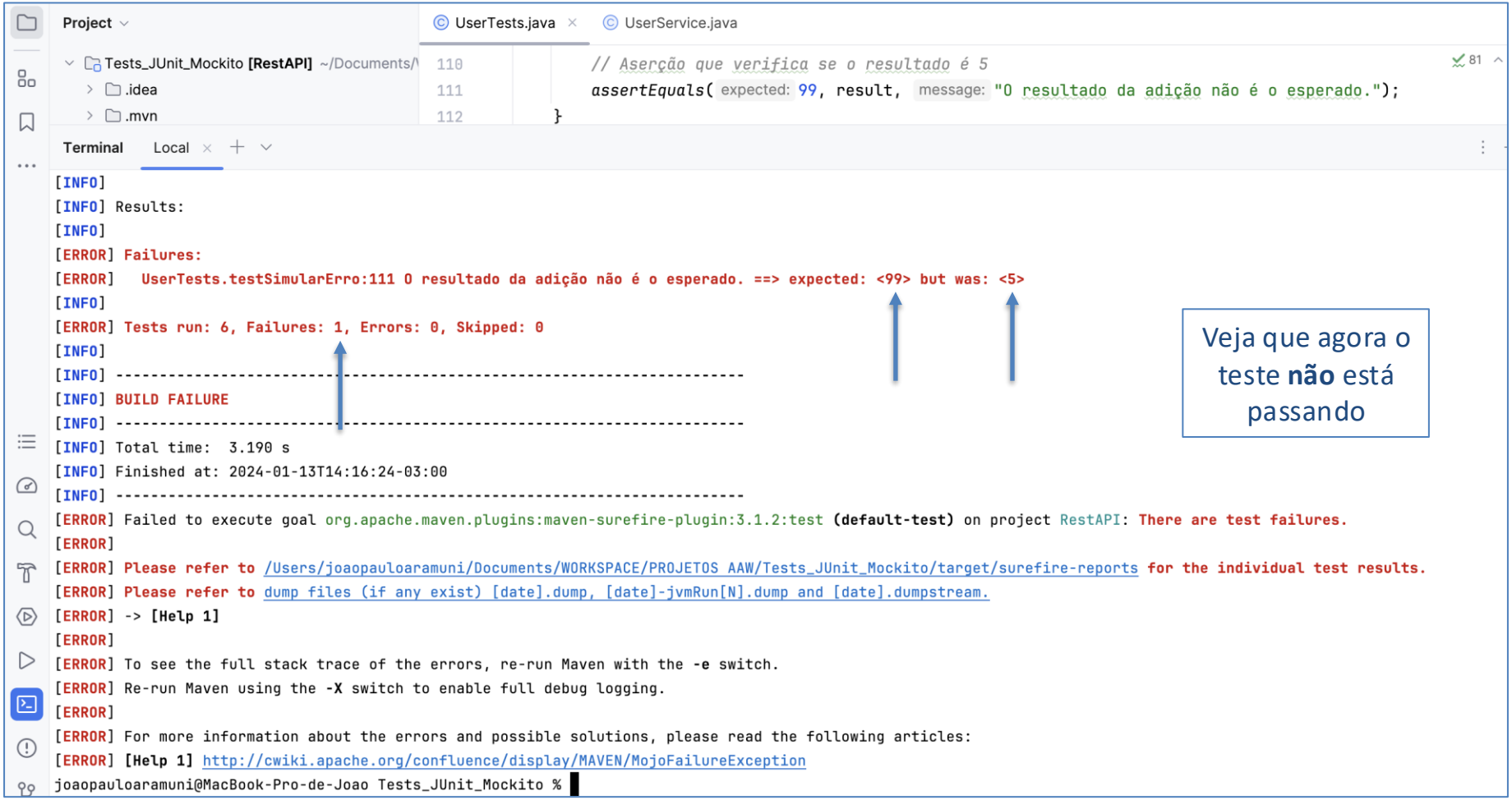


- **Forçando um erro**:
  - Veja que por enquanto o teste está passando:
  - testSimularErro()



- **Forçando um erro**:
  - Altere o resultado esperado de 5 para 99:
  - testSimularErro()




Project
Tests_JUnit_Mockito [RestAPI] ~/Documents/
.idea
.mvn
UserTests.java
UserService.java
110 // Aserção que verifica se o resultado é 5
111 assertEquals(expected: 99, result, message: "O resultado da adição não é o esperado.");
112 }
81
Terminal
Local
[INFO]
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR] UserTests.testSimularErro:111 O resultado da adição não é o esperado. ==> expected: <99> but was: <5>
[INFO]
[ERROR] Tests run: 6, Failures: 1, Errors: 0, Skipped: 0
[INFO]
[INFO] --
[INFO] BUILD FAILURE
[INFO] --
[INFO] Total time: 3.190 s
[INFO] Finished at: 2024-01-13T14:16:24-03:00
[INFO] --
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.1.2:test (default-test) on project RestAPI: There are test failures.
[ERROR]
[ERROR] Please refer to /Users/joaopauloaramuni/Documents/WORKSPACE/PROJETOS AAW/Tests_JUnit_Mockito/target/surefire-reports for the individual test results.
[ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
joaopauloaramuni@MacBook-Pro-de-Joao Tests_JUnit_Mockito %
Veja que agora o teste não está passando

Sumário

- Testes de resiliência dos serviços
  - Teste unitário
  - **Teste de desempenho**
  - Teste de carga

Testes de resiliência dos serviços

- **Teste de desempenho**
  - **Objetivo**: Avaliar a capacidade do sistema em atender aos requisitos de desempenho.
  - **Foco**: Medir e analisar métricas como tempo de resposta, taxa de transferência, latência e uso de recursos sob diferentes condições.
  - **Cenários**: Geralmente, testa cenários específicos de uso, como a resposta do sistema durante picos de carga ou períodos de atividade intensa.
  - **Ferramentas**: Pode envolver ferramentas especializadas, como Apache JMeter, Gatling, ou locais.

Testes de resiliência dos serviços

- **Teste de desempenho**
  - Tanto o teste de desempenho quanto o teste de carga são cruciais para garantir que um sistema seja robusto, escalável e capaz de lidar com diferentes condições de uso.
  - Eles são especialmente relevantes em ambientes onde a performance é crítica, como aplicações web, sistemas de comércio eletrônico e serviços online.





Sumário

- Testes de resiliência dos serviços
  - Teste unitário
  - Teste de desempenho
  - **Teste de carga**

Testes de resiliência dos serviços

- **Teste de carga**
  - **Objetivo**: Avaliar o comportamento do sistema sob cargas específicas e identificar seu limite de capacidade.
  - **Foco**: Aplicar cargas graduais ou aumentar a carga de trabalho até que o sistema atinja seus limites, observando como ele se comporta e se mantém estável.
  - **Cenários**: Geralmente, simula um grande número de usuários ou transações para entender como o sistema escala e se mantém eficiente.
  - **Ferramentas**: Pode envolver ferramentas semelhantes às usadas em testes de desempenho, mas com ênfase em simulações de carga pesada.

Testes de resiliência dos serviços

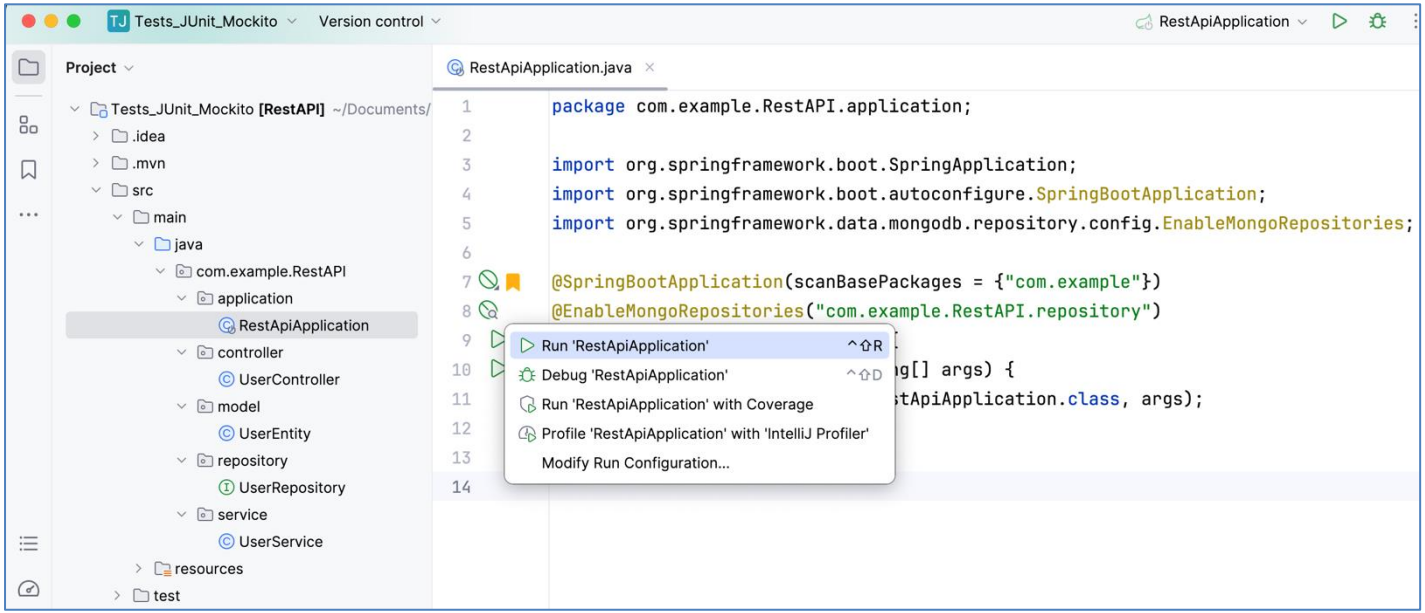
- **Ferramentas**
  - Conhecer ferramentas de monitoramento de performance e teste de aplicações é vital para que o desenvolvedor consiga se seniorizar mais rapidamente na carreira.

  - Vejamos três ferramentas muito úteis para este fim.

- **Ferramentas: JConsole**
  - **Propósito**: O JConsole é uma ferramenta de monitoramento e gerenciamento que faz parte do conjunto de ferramentas do JDK (Java Development Kit). Ele permite monitorar e gerenciar aplicações Java em execução na Máquina Virtual Java (JVM).
  - **Recursos**: Oferece informações detalhadas sobre a utilização de recursos da JVM, como CPU, memória, threads e gerenciamento de classes. Também permite a execução de operações de diagnóstico e ajuste de configurações em tempo real.
  - **Uso**: É útil para monitorar o desempenho de uma aplicação Java em um ambiente de desenvolvimento, diagnóstico de problemas e ajuste de parâmetros da JVM.

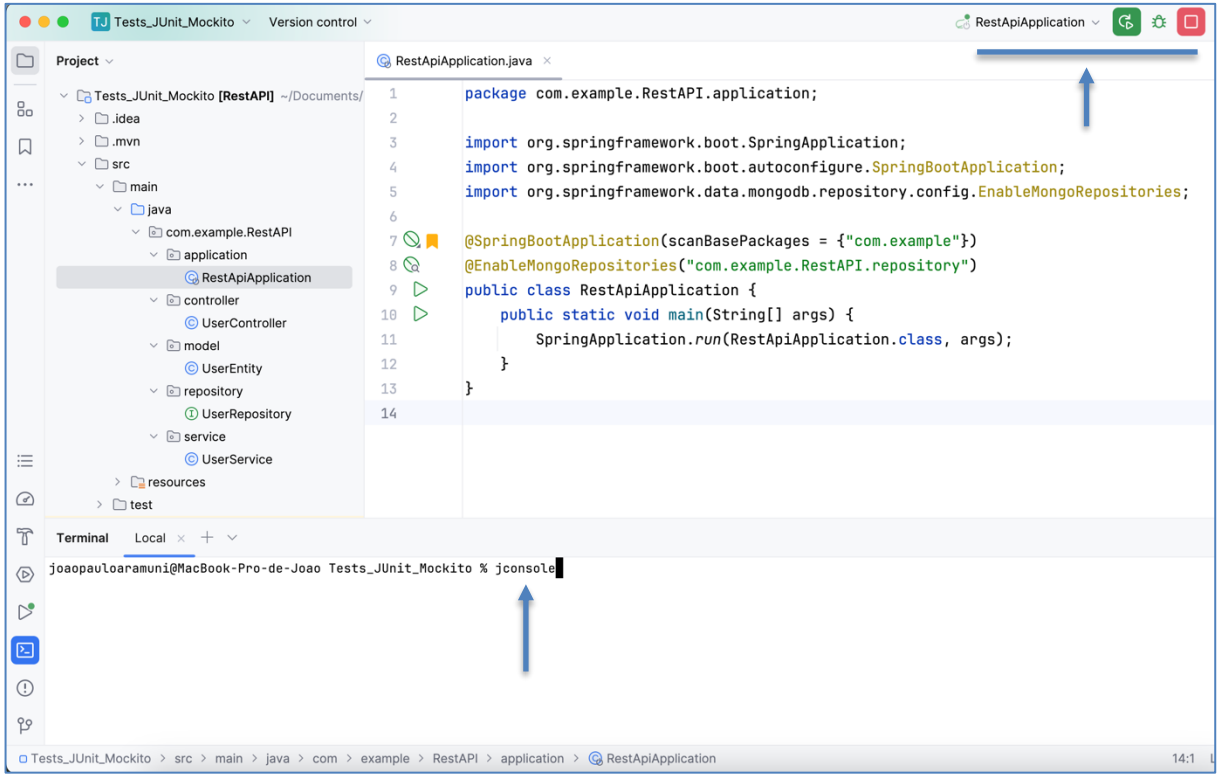


- **Ferramentas: JConsole**
  - https://docs.oracle.com/javase/8/docs/technotes/guides/management/jconsole.html
  - Teste o JConsole no terminal do seu IntelliJ IDEA.
  - Primeiramente, execute a sua aplicação Java.



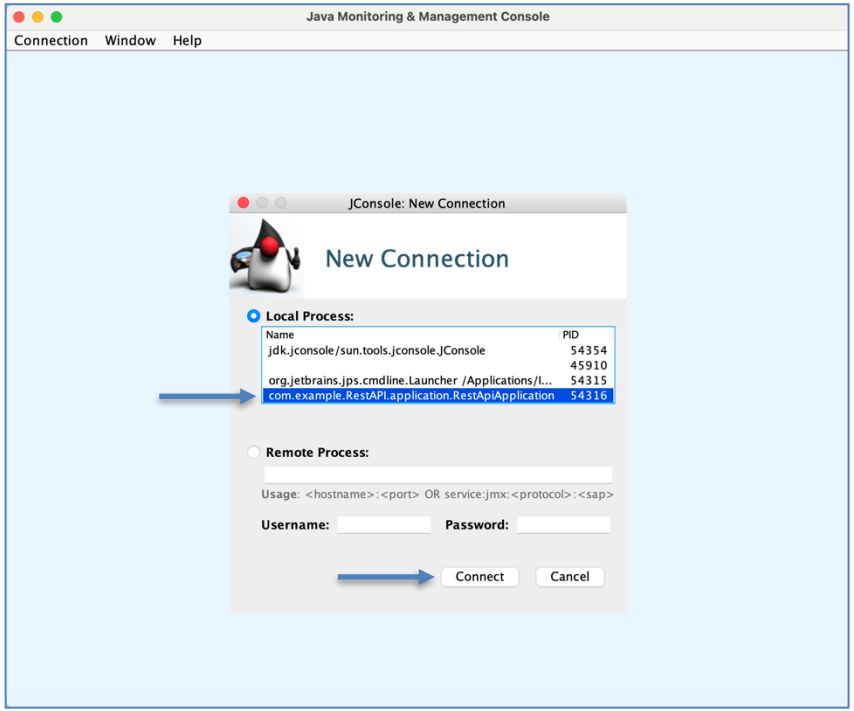


- **Ferramentas: JConsole**
  - Com a aplicação Java em execução, vá até o terminal e digite **jconsole**.



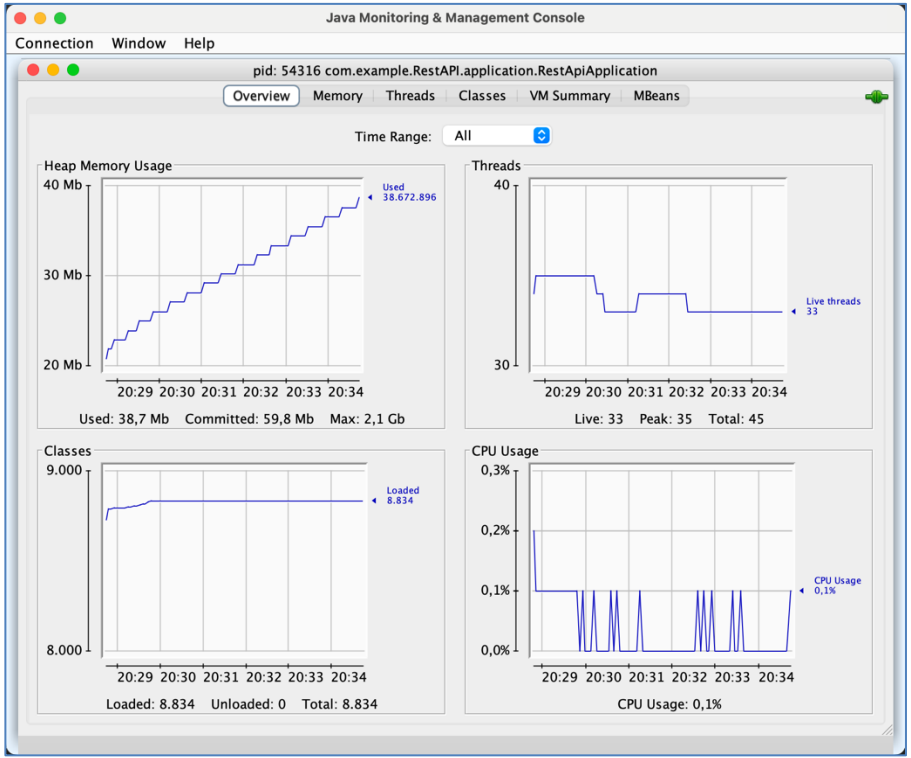


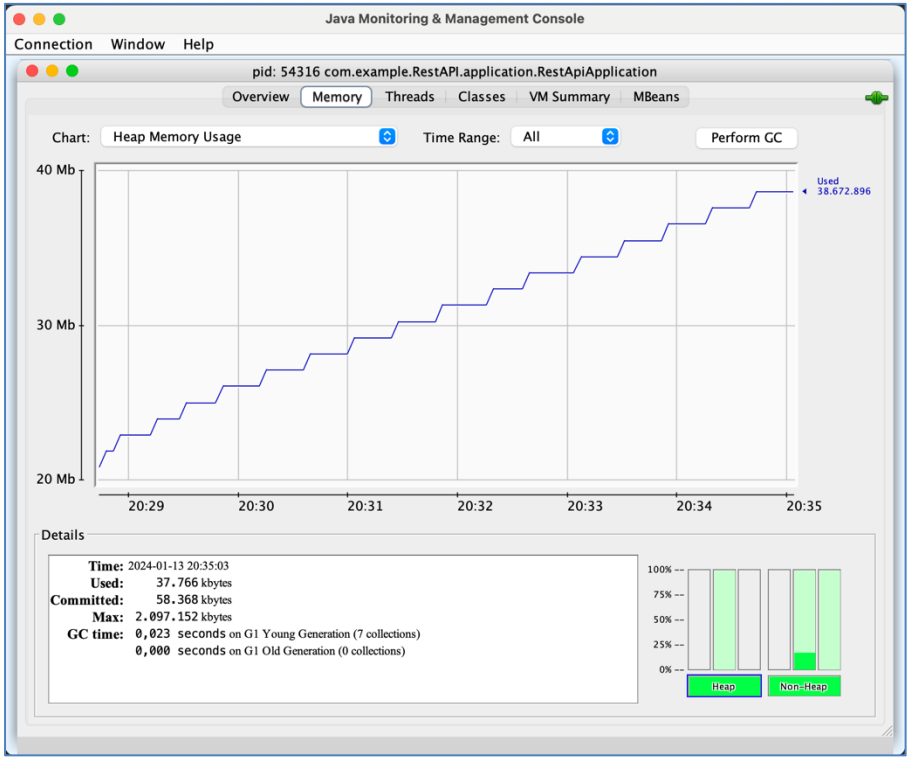
- **Ferramentas: JConsole**
  - Java Monitoring & Management Console
  - Selecione o processo referente à sua aplicação Java que deseje monitorar.



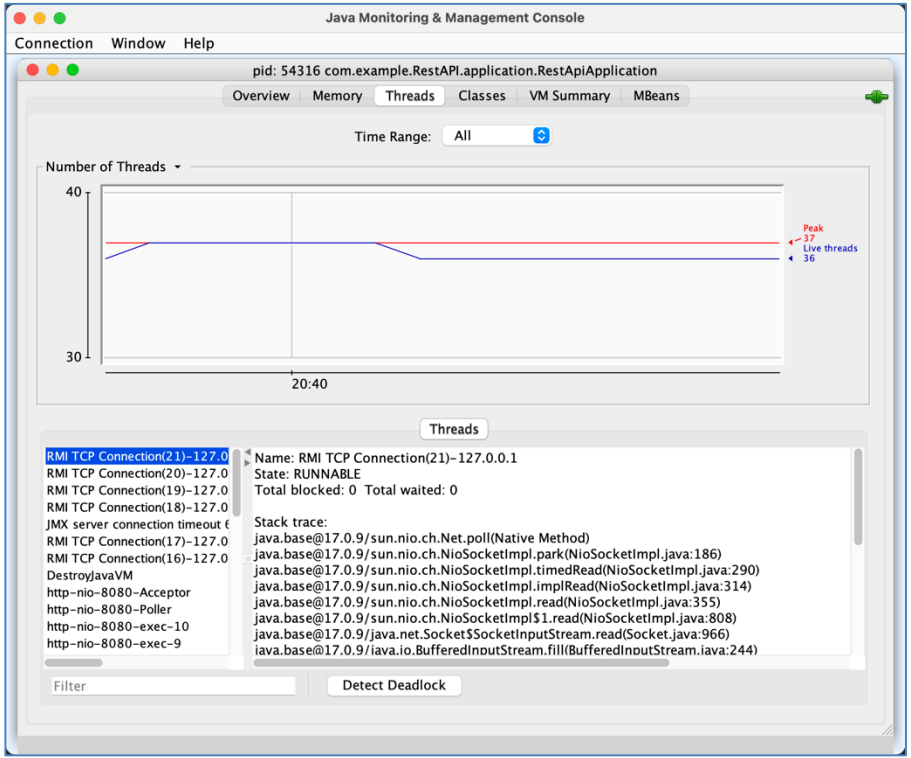


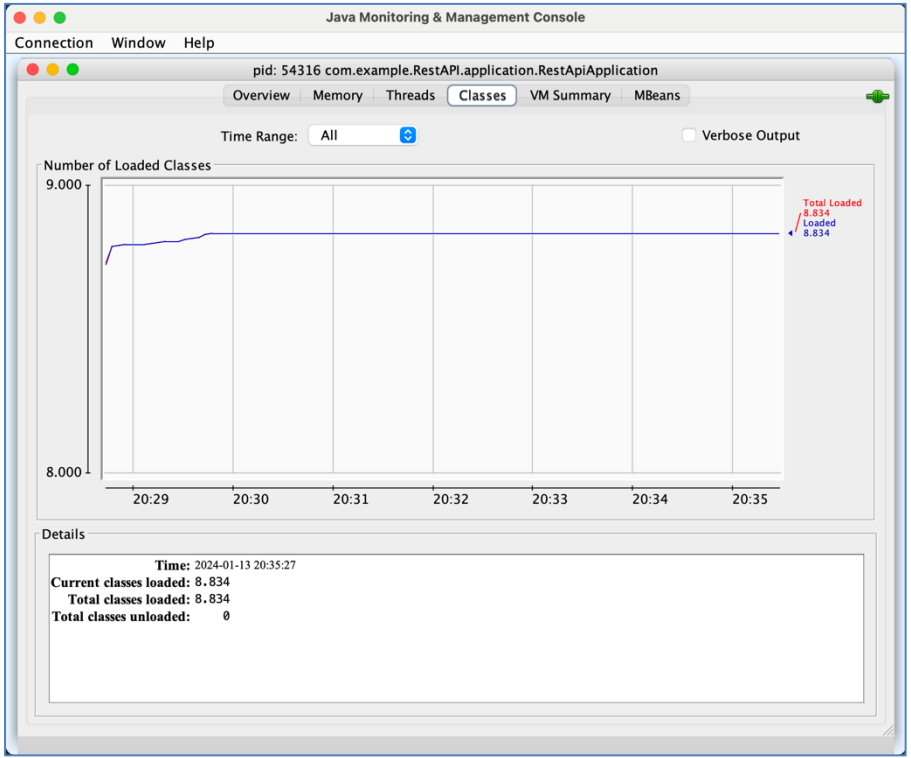
- **Ferramentas: JConsole**
  - Java Monitoring & Management Console



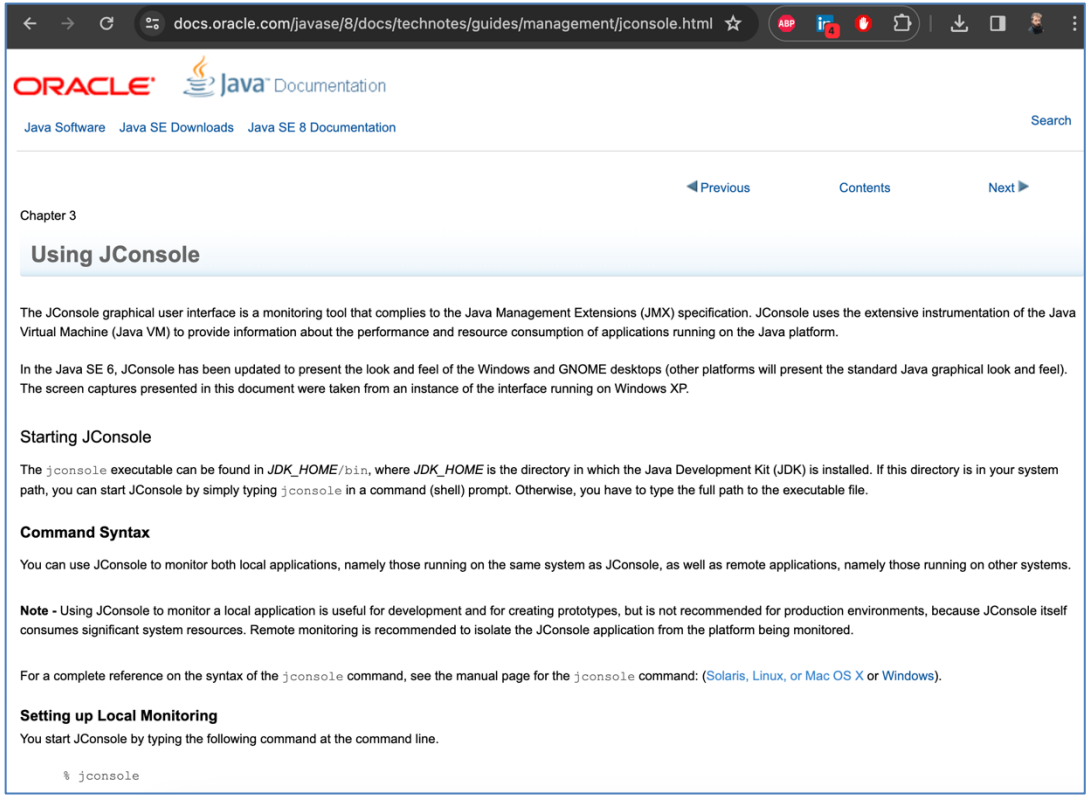


- **Ferramentas: JConsole**
  - Java Monitoring & Management Console





- **Ferramentas: JConsole**
  - https://docs.oracle.com/javase/8/docs/technotes/guides/management/jconsole.html





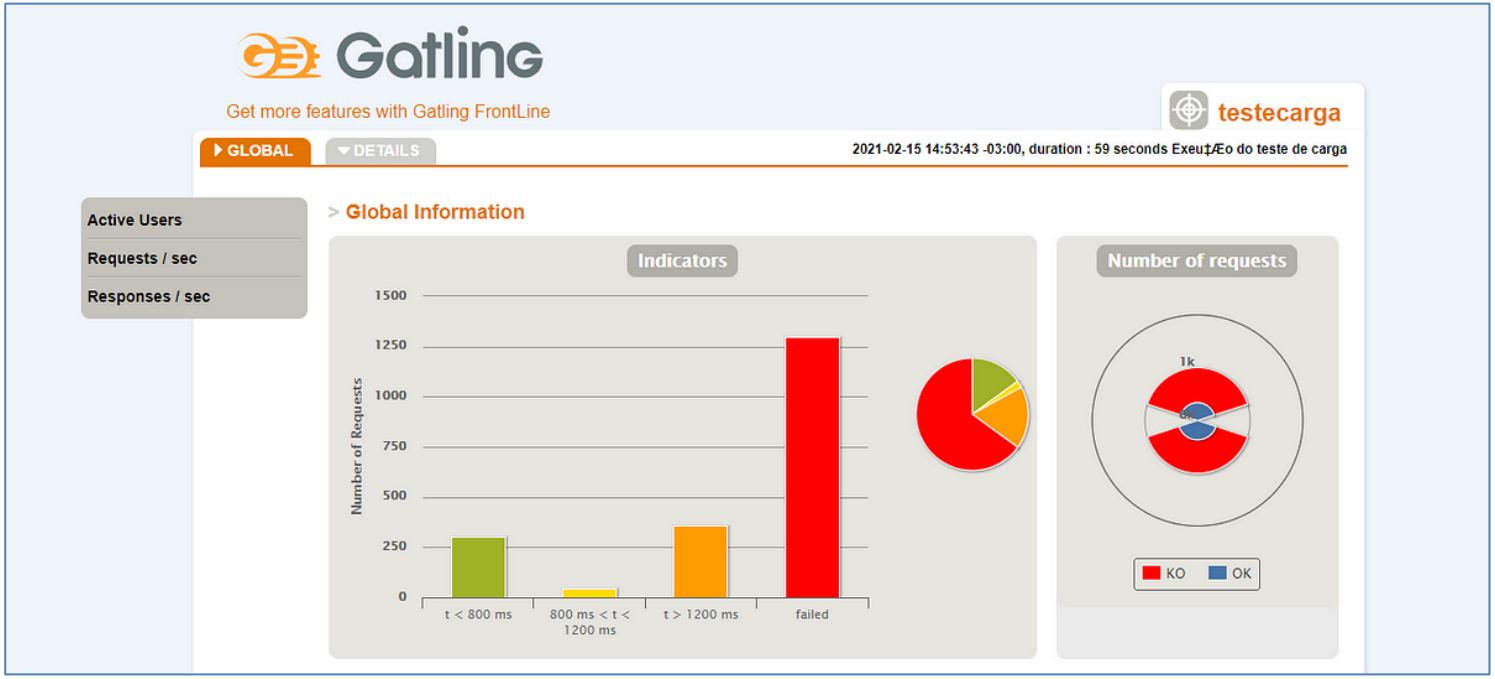
- **Ferramentas: Gatling**
  - **Propósito**: O Gatling é uma ferramenta de teste de desempenho e carga projetada para simular usuários interagindo com uma aplicação. Ele gera carga simulada para avaliar o desempenho da aplicação sob diferentes condições.

  - **Recursos**: Permite a criação de simulações realistas, oferece relatórios detalhados sobre o desempenho, suporta execução distribuída e fornece métricas como tempos de resposta, taxas de erro e transferência de dados.

  - **Uso**: É utilizado para testes de desempenho, avaliação do comportamento da aplicação sob carga e identificação de possíveis gargalos.



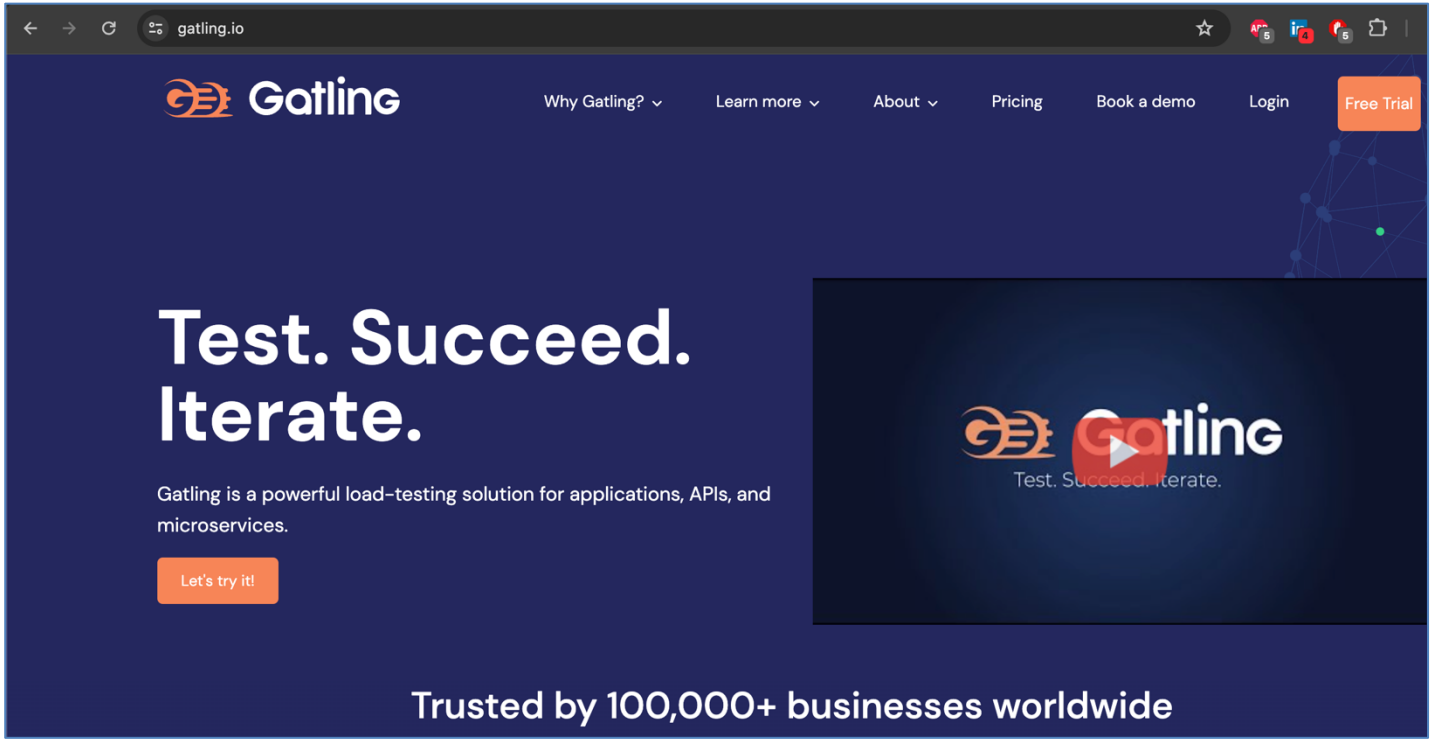
- **Ferramentas: Gatling**
  - https://gatling.io/





- **Ferramentas: Gatling**
  - https://gatling.io/







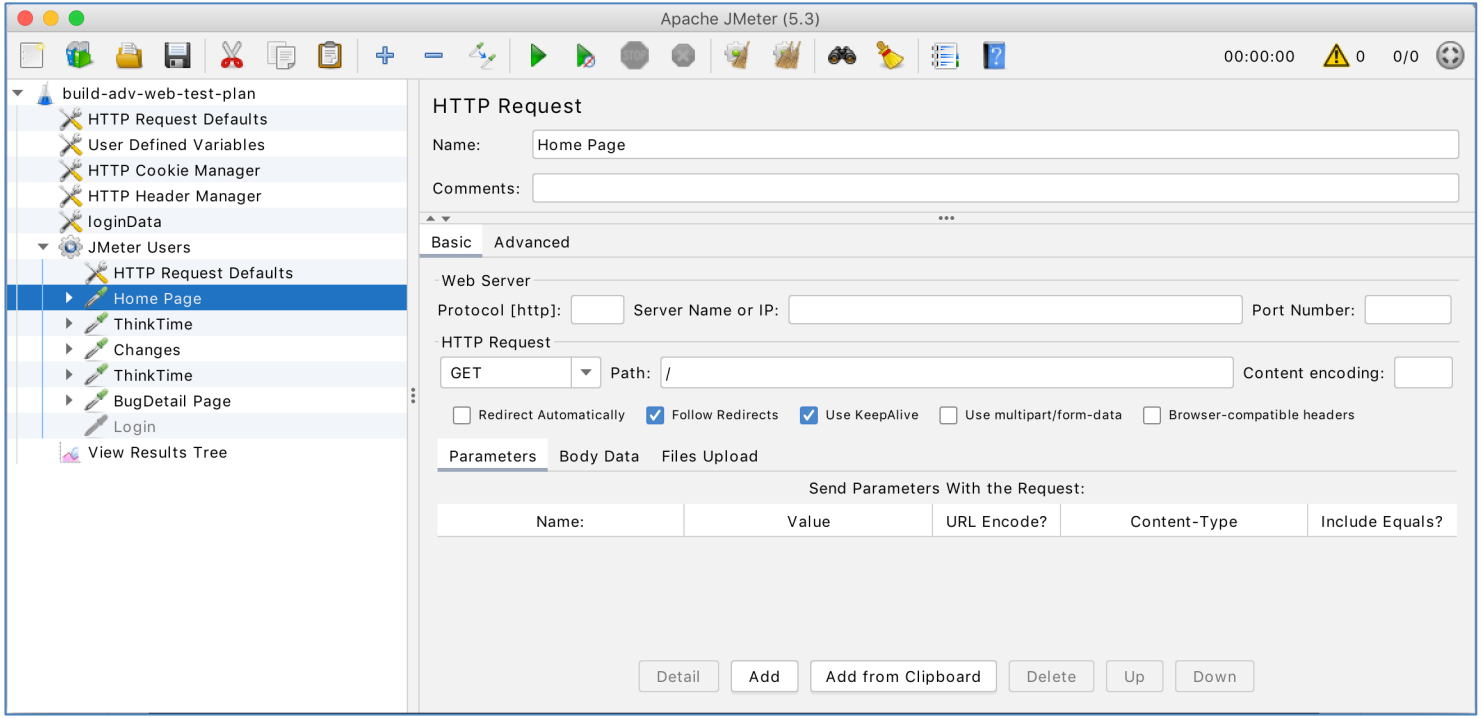
- **Ferramentas: JMeter**
  - **Propósito**: O Apache JMeter é uma ferramenta de teste de carga e desempenho projetada para medir o desempenho de serviços web, bancos de dados e outros tipos de serviços. Ele simula múltiplos usuários virtuais para avaliar o comportamento da aplicação.

  - **Recursos**: Suporta vários protocolos, como HTTP, JDBC, SOAP, entre outros. Oferece recursos para criar cenários complexos de teste, gerar relatórios detalhados e medir métricas de desempenho.

  - **Uso**: É amplamente utilizado para realizar testes de carga, testes de stress e avaliação de desempenho em diferentes tipos de sistemas.

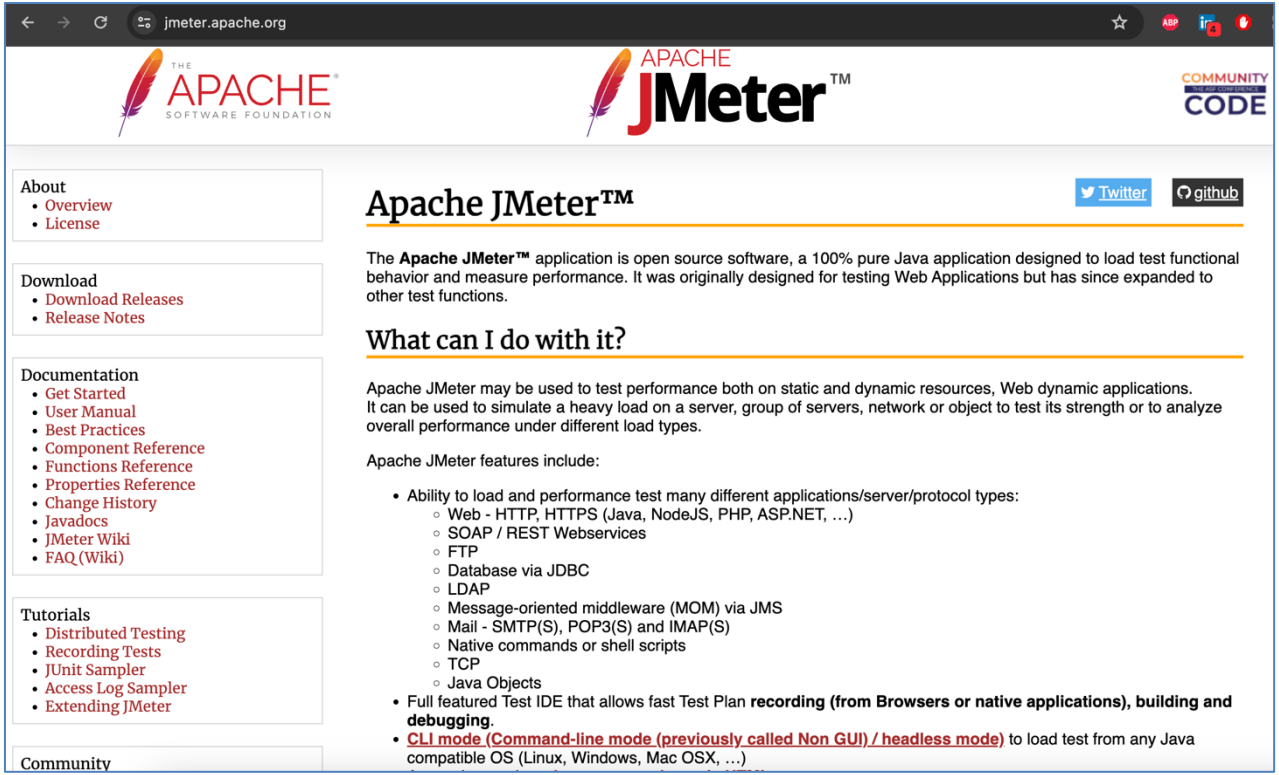


- **Ferramentas: Jmeter**
  - https://jmeter.apache.org/





- **Ferramentas: Jmeter**
  - https://jmeter.apache.org/





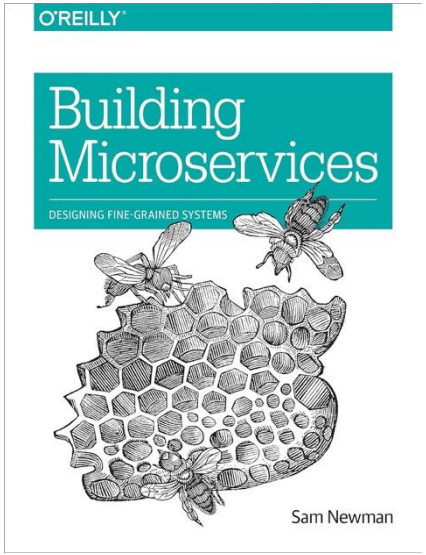
Testes de resiliência dos serviços

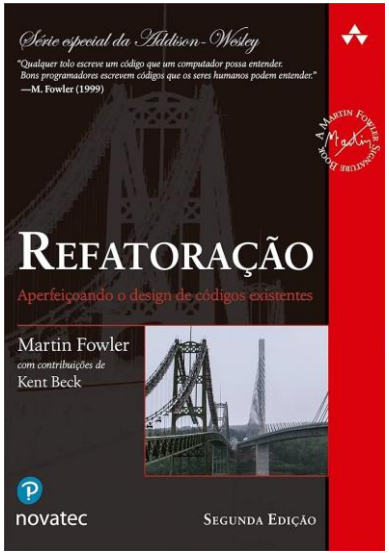
- **Ferramentas**
  - Enquanto o JConsole é mais focado em monitoramento e gerenciamento de aplicações em tempo real na JVM, Gatling e JMeter são ferramentas específicas para realizar testes de desempenho e carga em aplicações.

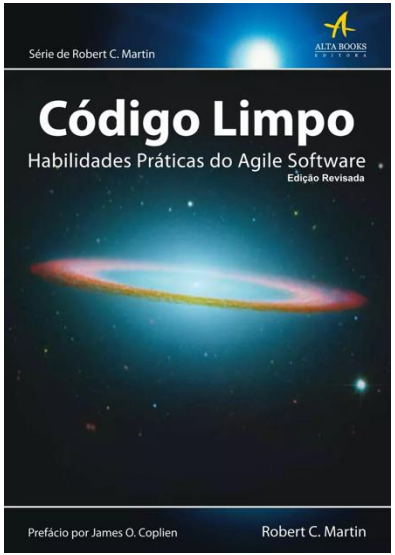
  - A escolha de cada uma dessas ferramentas dependerá dos requisitos específicos do projeto. É importante que o time avalie o *trade-off* de cada uma ou até mesmo a necessidade de utilizá-las em conjunto.

Referências básicas

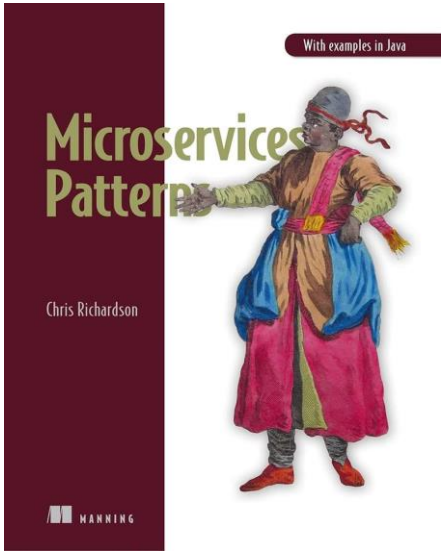


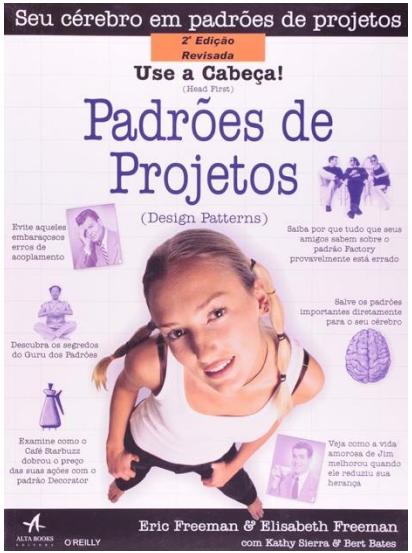


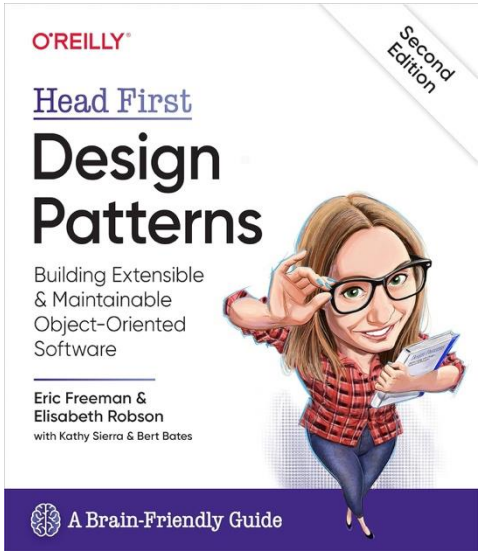


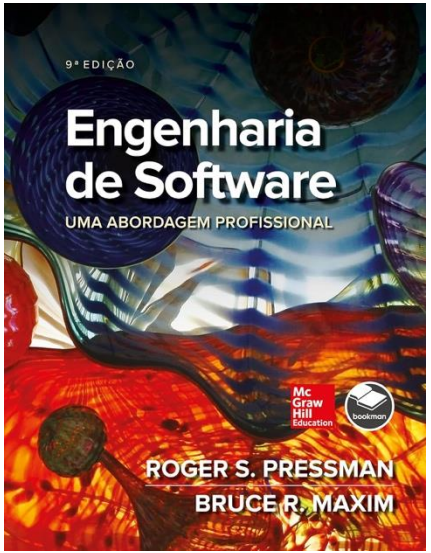


Referências complementares

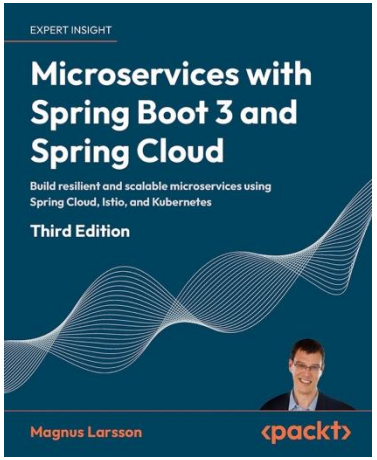




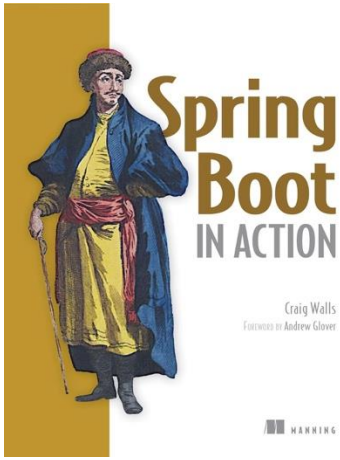


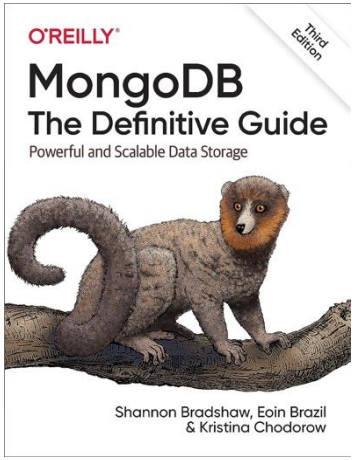


Outras referências

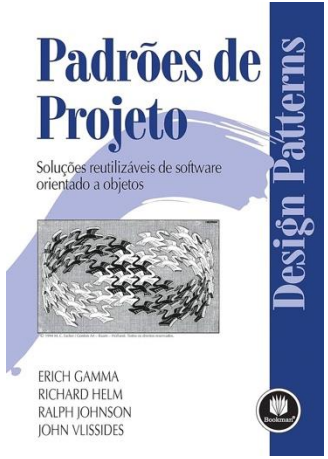












Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com





GitHub LinkedIn Lattes

...